



AALBORG UNIVERSITY

STUDENT REPORT

ED5-1-E17

Distributed Speech Recognition System

- Using Artificial Neural Networks -

Students:

Stefan Birs

Tiberiu-Ioan Szatmari

Kamran Thomas Alimaghani

Supervisors:

Daniel Ortiz-Arroyo

D. M. Akbar Hussain

Electronics and Computer Engineering
February 19, 2018



The Faculty of
Engineering and Science
Aalborg University
<http://www.en.aau.dk>

AALBORG UNIVERSITY

STUDENT REPORT

Title:

Distributed Speech Recognition System

Theme:

Machine Learning & Distributed Systems

Project Period:

Fall Semester 2017

Project Group:

ED5-1-E17

Participant(s):

Stefan Birs

Tiberiu-Ioan Szatmari

Kamran Thomas Alimagham

Supervisor(s):

Daniel Ortiz-Arroyo

D. M. Akbar Hussain

Copies: 1**Page Numbers:** 71**Date of Completion:**

February 19, 2018

Abstract:

This report goes into detail on the implementation of a distributed speech recognition system using artificial neural networks. A historical overview is presented on the different methods used in machine learning. Furthermore, neural network fundamentals for achieving a practical model for speech recognition are explained. Various pre-processing algorithms were developed, one of which being a .flac to .wav file converter used to create larger data sets for training. The Mel Frequency Cepstrum was used as a filter and pre-processing technique. All of the above mentioned steps were used to pursue a neural network with a bi-directional LSTM layer and fully connected layers. Following, a client-server framework was developed to distribute the obtained model. The report encapsulates the entire process for achieving a fully operational system.

Contents

Preface	vii
Acronyms	viii
1 Introduction	1
2 Problem Description	3
2.1 Tools	3
2.2 Problem Delimitation	4
3 Speech Processing	5
3.1 Classical Overview	5
3.2 Signal Processing	6
3.2.1 Pre-processing	7
3.2.2 Fourier Transform	7
3.2.3 Spectrogram	8
3.3 Modern Analysis	8
3.3.1 Mel Frequency Cepstrum Coefficients	8
3.3.2 Connectionist Temporal Classification Loss Function	9
4 Machine Learning Theory	10
4.1 Historical Overview	10
4.2 Artificial Neural Networks	11
4.2.1 What is an Artificial Neural Network?	11
4.2.2 Artificial Neuron	11
4.3 Types of Artificial Neural Networks	12
4.3.1 Feed Forward Network	12
4.3.2 Deep Recurrent Neural Network	13
4.3.3 Long Short-Term Memory	13
4.3.4 Bi-directional Neural Networks	16
4.4 Types of Regulation Techniques	18
4.4.1 Dropout	18
4.4.2 Batch Normalisation	19

5	Machine Learning Platform	20
5.1	TensorFlow	20
5.1.1	Tensors	20
5.1.2	TensorFlow Core	21
5.1.3	tf.train API	21
5.1.4	tf.estimator API	22
5.2	Batch Size, Dropout and Learning Rate Comparison	23
5.2.1	Batch Size Test	24
5.2.2	Dropout Test	26
5.2.3	Learning Rate Test	27
6	Model Development	30
6.1	Neural Network Comparison	30
6.2	Model Description	33
7	Network Framework	37
7.1	Virtual Private Network (VPN)	37
7.2	Sockets	38
8	Implementation	39
8.1	Data Mining	39
8.2	DSR System Implementation	41
8.3	Testing	43
9	Discussion	44
9.1	Future Work	45
10	Conclusion	47
	Bibliography	48
11	Appendix	50
A	MATLAB	51
A.1	Waveform, FFT and Spectrogram Script	51
B	Batch Size, Dropout and Learning Rate Comparison Data	53
B.1	Batch Size Test	53
B.2	Dropout Test	56
B.3	Learning Rate Test	59
C	Model Development Comparison Data	62

D Data Mining Code	66
D.1 FlacToWavPy	66
D.2 txtFileMaker	68

Preface

The project entitled Distributed Speech Recognition System was made by three students from the Electronics and Computer Engineering programme at Aalborg University Esbjerg, for the P5 project during the fifth semester. All of the software written for this project will be referenced on the GitHub project repository.

We would like to thank our supervisors, Daniel Ortiz-Arroyo and Akbar Hussain, for the excellent supervision and the help they provided in the initial phases of choosing a project.

We would like to also thank our semester coordinator, Simon Pederson, and Daniel Ortiz-Arroyo for bestowing upon us the necessary hardware that was used to achieve our goals, without which this project couldn't have been made.

Aalborg University, February 19, 2018

Stefan Bîrs

<sbars15@student.aau.dk>

Tiberiu-Ioan Szatmari

<tszatm15@student.aau.dk>

Kamran Thomas Alimagham

<kalima15@student.aau.dk>

Acronyms

TF	TensorFlow
DSR	Distributed Speech Recognition
MFCC	Mel-Frequency Cepstral Coefficients
GPU	Graphics Processing Unit
CTC	Connectionist Temporal Classification
DTW	Dynamic Time Warping
VQ	Vector Quantization
ANN	Artificial Neural Networks
AN	Artificial Neuron
RNN	Recurrent Neural Network
FC	Fully Connected
LSTM	Long Short Term Memory
BiRNN	Bi-directional Recurrent Neural Network
BN	Batch Normalisation
API	Application Programming Interface
WER	Word Error Rate
VPN	Virtual Private Network
ASR	Automatic Speech Recognizer
IR	Information Retrieval

Chapter 1

Introduction

The scope of this project will be on implementing a distributed speech recognition (DSR) system. The main focus is to understand, design and implement an artificial neural network that can recognize human speech using deep learning methods. The toolkit that we will use for the machine learning part was developed by the Google Brain Team and it's now an open-source, software library, called TensorFlow (TF) [1]. At the time of this project, the newest update available is version 1.3.0.

Speech is probably the most important forms of communication. It is the most natural and efficient way to communicate with each other. Humans learn all the relevant skills during early childhood, without any instruction, and they continue to rely on speech communication throughout their life [2]. People also want to have a similar, natural and efficient form of communication with machines. Until recently, the idea of holding a conversation with a computer seemed pure science fiction.

However, the situation is changing, and quickly. A growing number of people now talk to their smartphones, asking them to set reminders, search for directions, or send email and text messages. Chief technology officer of © Nuance Communications, Vlad Sejnoha stated: "We're at a transition point where voice and natural-language understanding are suddenly at the forefront" [2].

Reading Guide

In Chapter 3 the pre-processing part of the project is covered. In it, detailed explanations of historical and modern approaches are taken into consideration. Chapter 4 presents an introduction to the field of artificial neural networks and displays the learning process by gradually introducing more complex types of networks. Alongside the theoretical part, a practical introduction to TensorFlow is given in Chapter 5. Chapter 6 shows the difference between three working models, detailing the advantage of a bi-directional network model. Chapter 7 talks about the assembly and implementation of the distributed system. Chapter 8 and 9 provide a summary of the findings from the

project, an analysis of future work and a closing argument. All the code and scripts used for the project can be found on the GitHub repository referenced here: [3].

Chapter 2

Problem Description

As mentioned above, the final goal of the project is to develop a distributed speech recognition (DSR) system. In order to achieve this goal we need to gain a fundamental understanding in the field of machine learning and computer networks.

2.1 Tools

The following information is a selection of the relevant software and hardware tools used to implement graphics processing unit (GPU) accelerated training:

Software Tools

- TensorFlow 1.3.0
- Python 3.6
- Visual Studio 2015
- cuDNN 8.0
- MATLAB R2017b

Hardware Tools

- Supermicro C7Z87 motherboard
- Intel I7 4770 CPU
- NVIDIA Geforce GTX 1070 GPU
- 8GB DDR3 RAM
- 320GB hard drive
- Supermicro PWS-502-PQ 500W

Setup

As seen in the listing above, the Nvidia GTX 1070 was the main unit used for training neural networks. Supporting 1920 CUDA cores, a standard memory configuration of 8 GB GDDR5 and a maximum power consumption of 150W [4]. It was added to a computer with an Intel Core I7-4770 CPU and 8 GB of RAM memory. The operating system was Windows 7 Enterprise.

2.2 Problem Delimitation

To ensure that the project meets the desired scope, specific success criteria were formed which will be evaluated during the final stages of the report. The following list of requirements is the desired success criteria, where the focus will be on reaching the given goals:

- Data mining
- Choose a filter
- Design a deep neural network model
- Train for the English language
- Create a client-server framework
- Implement the distributed model

A key component of neural networks is data. To increase their performance, using a large data set is necessary. As open-source, compatible, training data sets are hard to come by, data mining algorithms were created. This will allow the available data to fit the needs of the developed model.

Any real-life sequence of recorded speech has some amount of noise, making it harder for the neural network to understand spoken words. This can lead to errors that alter the words beyond recognition. To ameliorate this problem, a filter that mimics the human ear is the best fit for such a project.

Along the report, a historical analysis of different knowledge-based solutions to the recognition of human speech is made. By studying the existing literature and comparing open-source projects, a neural network can be built, with the scope of reducing the error rate and training time as much as possible.

A client-server set-up allows for running different parts of the DSR system, on multiple, internet connected machines. This allows for the more demanding tasks to be processed remotely, minimizing the impact for the end-user.

Chapter 3

Speech Processing

Speech is an effortless and highly efficient form of communication. Continuous speech recognition software is readily available from stores and it allows the user to interact with its surrounding without the need for written commands [5, p. 396]. Amazon's Alexa [6] and Microsoft's Cortana [7] are perfect examples of home companions that are able to perform basic tasks received via voice commands.

As speech recognition increases in accuracy, this technology will make the leap from being annoying to use, to becoming the main way we interact with computers. This idea is further backed up by the fact that people tend to speak much faster than they are able to type [8]. In some cases, speech can be three times faster in conveying information than typing, and for professional debaters, speaking can be even six times faster.

3.1 Classical Overview

Words are carried as sound waves, which are analog signals. In order for speech to be processed, it needs to be run through a signal processor that selects the frequencies and amplitude of the signal. Further on, the signal is mapped to individual sound units called phones. These phones need to be identified and grouped in such a way to ensure that each word has a different phonetic structure, otherwise, words would be impossible to distinguish from one another.

Table 3.1: Phones.

[ay]	<u>i</u> ris
[b]	<u>b</u> in
[er]	<u>b</u> ir <u>d</u>
[l]	<u>l</u> ip
[p]	<u>p</u> in
[th]	<u>t</u> h <u>i</u> n

Phones can have different sounds depending on the context. For example, the phone **th** in the word *three* has a different sound to **th** in *then*. To overcome these different variations of the same phones, it is better to abstract the phones into a generalized grouping called a **phoneme**. Phonemes are written in-between forward slashes (/th/) and they will have a specific pronunciation for each sound, depending on the context. These phonemes are used as a transitional layer when trying to convert speech to text and vice versa, when speech needs to be synthesized.

3.2 Signal Processing

Sound waves that carry human speech are variations in air pressure. The key components of a sound wave are its **amplitude**, which measure the intensity of the sound and the **frequency**, which describes the rate at which the amplitude varies over time. When speaking into a microphone, the change in air pressure causes the diaphragm to oscillate. The size of the oscillations is directly proportional to the amplitude of the signal, while the rate at which the diaphragm oscillates represents the rate at which the air pressure changes. At specific time intervals, the signal can be sampled and the data can be used in a wide array of digital signal processing tools. For example, the digital signal can be plotted as a two-dimensional plot, where the y-axis defines the amplitude and the x-axis shows the time. The frequency can be easily be determined from the plot as the number of cycles that the signal does per second.

Although a visual difference can be seen when plotting vowels and consonants, a visual inspection of the waveform will not reveal any critical information, making it impossible to see all the phonemes.

To feed this information to the neural network, some preprocessing and signal manipulation is required so the input can be understood and passed through the neurons. These steps are expanded upon in the following subsections.

3.2.1 Pre-processing

No matter how powerful and how advanced a computer is, it still works in discrete time. In order to represent an analog signal on the computer, it has to be sampled according to the Nyquist sampling theorem. This theorem states that the sampling frequency has to be at least double the frequency of the original signal in order to replicate the original signal without aliasing. For a better understanding of the process, a visual representation of the word "Hello" as it is formed for the input of the neural network is shown in the upper part of Figure 3.1.

3.2.2 Fourier Transform

To analyze a discrete signal we use the Fourier transform. This allows us to convert the signal in the frequency domain and deconstruct it in a series of simple sine waves. These sine waves have specific amplitudes and frequencies and while plotting the FFT of the signal, the dominant waves can be identified by the intensity of the amplitude.

In the lower part of Figure 3.1, a visual representation of the FFT is shown. It can be seen that there are a lot of frequencies that make up the word "Hello". The dominant ones are in the range of 500Hz. Considering that this recording has a minimum amount of noise, we have to consider the full range of the frequencies, from zero and up to 3500 Hz.

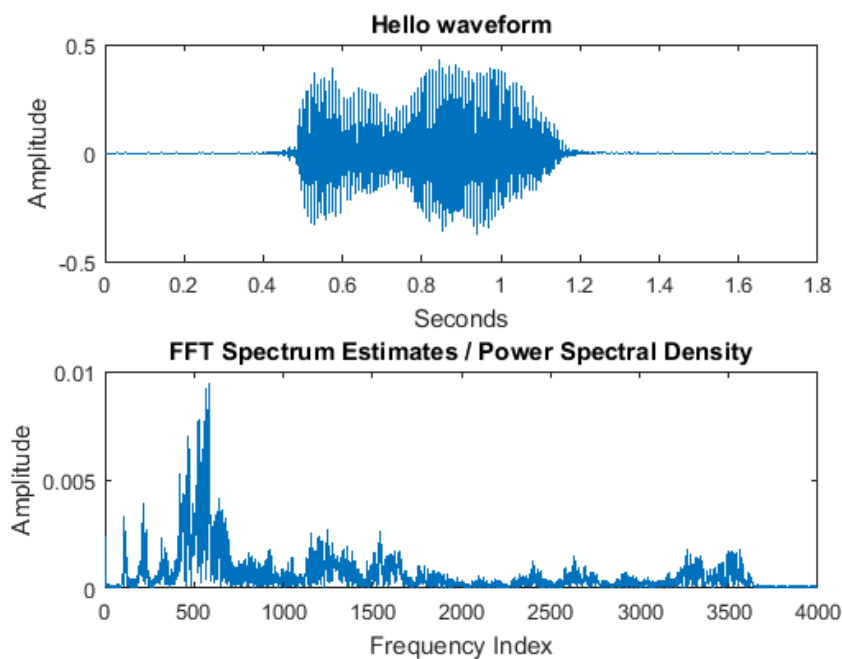


Figure 3.1: Waveform and FFT computed for the word "Hello".

3.2.3 Spectrogram

The spectrogram of the signal was computed from the FFT and it shows a visual representation of the spectrum of frequencies marked by the intensity of the color. After the spectrogram is sliced into 25 ms chunks, it is feed as the input to the network. A neural network can find patterns in this data more easily than raw sound waves because certain patterns can be distinguished in the graph. Such a spectrogram can be seen in Appendix A, Figure A.1.

3.3 Modern Analysis

In the last decade, the state-of-the-art speech recognition models were phonetic-based approaches. Acoustic models with generative probabilistic models for sequential data, like the Hidden Markov Model (HMM) require both text and speech data and, furthermore a word to phoneme dictionary. Even though these models can be made more accurate when speech data is correlated with phoneme transcriptions, this is a gargantuan task to be attempted by hand. This makes phoneme-level transcriptions less likely to be used on large data sets. Both speech recognizers of the past and today rely on segmenting sound waves, but word based processing is far more efficient in modern speech recognizer (SR) systems.

3.3.1 Mel Frequency Cepstrum Coefficients

Mel-frequency cepstral coefficients (MFCCs) are coefficients that collectively make up a Mel-frequency cepstral (MFC). They are taken from a cepstral representation of the audio clip by finding the spectrum of the spectrum of the original audio file. Compared to normal cepstrum, in MFC, the frequency bands are spaced at equal distances on the Mel scale. This mimics a humans response to sound better than a linearly represented frequency band [9].

Speech is a continuous action, where the present utterance will affect the future one. Combined with the fact that the English language is not a phonetic language means that a direct representation of each character to their sound is impossible. To overcome this problem, the neural network is trained on overlapping windows both before and after the current speech sample.

In the model developed by us, nine slices of audio are used before and after the current index, adding up to 19 time points for each window. With 26 cepstral coefficients, this gives 494 data points per 25ms observation, sampled at 16000Hz and seen in Listing 3.1 .

```

1 # Load wav files
2 fs, audio = wav.read(audio_filename)
3
4 # Get mfcc coefficients
5 orig_inputs = mfcc(audio, samplerate=fs, numcep=numcep)
6
7 # For each time slice of the training set, we need to copy the context this makes
8 train_inputs = np.array([], np.float32)
9 train_inputs.resize((orig_inputs.shape[0], numcep + 2 * numcep * numcontext))
10
11 for time_slice in range(train_inputs.shape[0]):
12     # Pick up to numcontext time slices in the past,
13     # And complete with empty mfcc features
14     need_empty_past = max(0, ((time_slices[0] + numcontext) - time_slice))
15     empty_source_past = list(empty_mfcc for empty_slots in range(need_empty_past))
16     data_source_past = orig_inputs[max(0, time_slice - numcontext):time_slice]
17     assert(len(empty_source_past) + len(data_source_past) == numcontext)

```

Listing 3.1: Example code to obtain MFCC features [10].

3.3.2 Connectionist Temporal Classification Loss Function

Discarding the use of phonemes for neural networks can be done by switching to an objective function that computes predictions at the character-level transcription.

Connectionist Temporal Classification (CTC) provides probabilities for multiple sets, encapsulating all possible character-level transcriptions of the speech sample. The network uses the objective function to choose the highest probability for a transcription and calculates the error rate. The error is generated by comparing the predicted result to the absolute value of the transcript before updating the weights of the network during training [11].

The character-level error used by the CTC loss function is different from the Levenshtein [12] word error distance that is used in traditional models. Considering that the scope of this project is limited to the English language, it is important to know that the character versus error rate will be quite different because English is a non-phonetic language. An in-depth analysis of the CTC inner works can be found in [13] and [14].

Chapter 4

Machine Learning Theory

The goal of this chapter is to understand different kinds of Artificial Neural Networks (ANNs), starting with a historical overview of older, but highly recognized machine learning techniques still used today. After the brief overview, ANN will be looked at in more detail and different configurations are shown. Leading on, a simple description on TensorFlow and respective APIs (with example code) is done in order to understand future code development. Finally, a small comparison study is presented to get a feeling for the different parameter values.

4.1 Historical Overview

The following section will encapsulate different methods that can be used in the field of speech processing and recognition. Starting with statistical models and mapping algorithms, further leading to knowledge-based systems that render greater efficiency when large data sets are available.

Hidden Markov Model

The forward and backward recursions used in HMM were created in 1960 by Ruslan L. Stratonovich [12]. It is the most successfully used pattern recognition technique used for speech recognition. In contrast to knowledge-based approaches, this model has a strong and secure mathematical foundation where a mathematical model is signalized on the Markov Model and a set of output distribution. Speech is divided into smaller audible samples where each sample represents a state in the Markov Model. By the probabilities of transition, there will be a shift from one state to another [15, p. 2][16].

Dynamic Time Warping

Dynamic Time Warping (DTW) is an algorithm that compares words with pre-given reference words. It measures the patterns between two sequences that will vary in time

or speed [16]. To output speech, the algorithm tries to alter the time variable of the unknown speech until it matches one of the reference words.

Vector Quantization

Vector Quantization (VQ) is used to map vectors from a big vector space to a finite region in space. It needs compact code-blocks for reference models and codebook searcher in place of more costly evaluation methods. Each word receives a VQ codebook that is based on repeated sequences of the word. The test data is evaluated on all codebooks and the automated speech recognizer picks the codebook with the lowest distance measured [16, p. 20-21].

4.2 Artificial Neural Networks

4.2.1 What is an Artificial Neural Network?

Artificial neural networks are nothing but a computerized representation of the human brain. They have the ability to acquire and maintain knowledge (information based) and can be defined as a set of processing units, represented by artificial neurons, inter-linked by a lot of interconnections (artificial synapses) [17, p. 5].

The human brain learns from its experiences, creating new neural pathways. These interconnected chains of neurons are stronger than others and can be modulated, or changed, following learning or during behavioral modifications. The strength of a neural connection is given by a weighted value, called synaptic weights.

4.2.2 Artificial Neuron

The artificial neuron is the processing unit of an artificial neural network, which is a simplified model of the biological neuron. This model was inspired by the analysis of how a cell membrane of a neuron generates and propagates electrical impulses (Hodgkin and Huxley 1952) [17, p. 11]. The purpose of artificial neurons is to simulate the basic function of biological ones, which are typically comprised of four parts:

1. Dendrites: Accept inputs
2. Soma (cell body): Process the inputs
3. Axon: Turn the processed inputs into outputs
4. Synapses: The electrochemical connections between neurons

Figure 4.1 illustrates the four main functions of a biological neuron, represented by the artificial one.

The artificial neurons used in artificial neural networks are non-linear, usually providing continuous outputs, and performing simple functions, such as gathering signals available on their inputs, assembling them according to their operational functions, and producing a response considering their innate activation functions [17, p. 11].

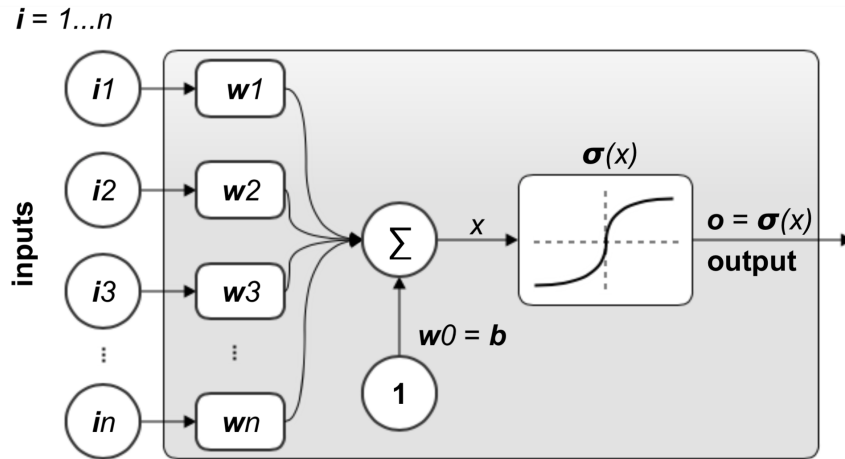


Figure 4.1: The artificial neuron.

Each neuron from a network can be implemented as shown in Fig. 4.1. The multiple input signals coming from the external environment are represented by the set $\{i_1, i_2, i_3, \dots, i_n\}$, analogous to the electrical signals received by the dendrites from the biological neuron.

Those inputs are then multiplied with a weight vector $\{w_1, w_2, w_3, \dots, w_n\}$. This is how each input gains a different relevance that will affect the output of the neuron. In Figure 4.1 it can be seen that the neuron computes a weighted sum of all its weights and biases, which then are passed onto the activation function to be mapped, analogous to the biological function of the Soma and Axon respectively. This process is repeated for each neuron found in any layer of a neural network.

4.3 Types of Artificial Neural Networks

4.3.1 Feed Forward Network

This type of simple neural network is comprised of one input layer and one neural layer, which also acts as the output layer. The flow of information is unidirectional, starting from the input layer and progressing to the output layer. As such, the number of outputs from the neural network will always coincide with the number of neurons

in the network. These networks are usually employed in pattern classification and linear filtering problems. The Perceptron and the ADALINE architectures are among the most recognized feed-forward neural networks and they work well with Hebb's rule and the Delta rule for training.

4.3.2 Deep Recurrent Neural Network

To explain how a deep recurrent neural network (RNN) looks like first we will see what a deep neural network is made of and after that, the recurrent part will be added to form a more complex and powerful network.

Networks with multiple layers have one or more hidden neural layers. Such a network will always have one input layer with multiple samples and a series of variable layers. In contrast to the simple feed-forward network, these hidden layers are not constrained to have the same size as the input layer. This means that the size can depend on the complexity of the problem being tackled by the network, as well as the quantity and quality of the available data. The last layer still occupies a double role as both a neuron layer and output layer.

Another key element is the fact that the outputs of the neurons are used for feedback inputs to the other neurons. This feature makes the network great for time-variant systems where the new outputs can benefit from the previous information.

4.3.3 Long Short-Term Memory

Long Short-Term Memory networks, for short "LSTMs", are a special case of the recurrent neural network, that is capable of learning long-term dependencies. Firstly used in 1997 by Sepp Hochreiter and Jürgen Schmidhuber [18] and refined by Felix Gers and his team [19]. Nowadays, after further refinement, LSTMs work incredibly well on various problems and give better results for most problems that require RNNs. They remember information for long periods of time by default, making them the perfect fit to avoid long-term dependency problems.

LSTMs have a chain-like structure of four layers, with repeating modules that interact in specific ways. For simplicity, in the following explanation of the internal states of an LSTM, the input X will represent the normal input to the network concatenated with the previous state, $X = x_t|h_{t-1}$.

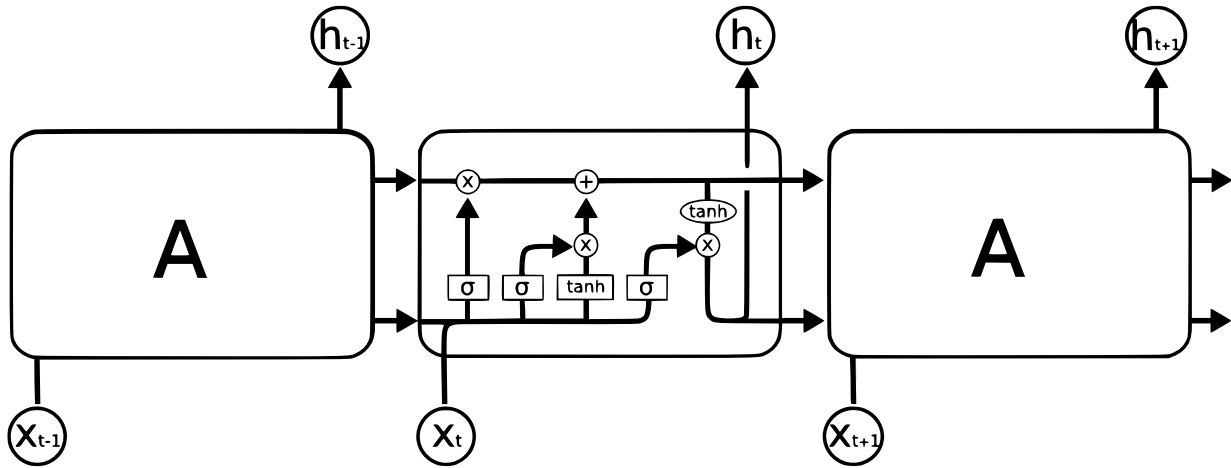


Figure 4.2: LSTM Diagram.

From Figure 4.2 [20], the lines represent vectors that pass from the output of the previous node to the input of the current one. The circle denotes point wise operations, and the rectangle represents trained network layers.

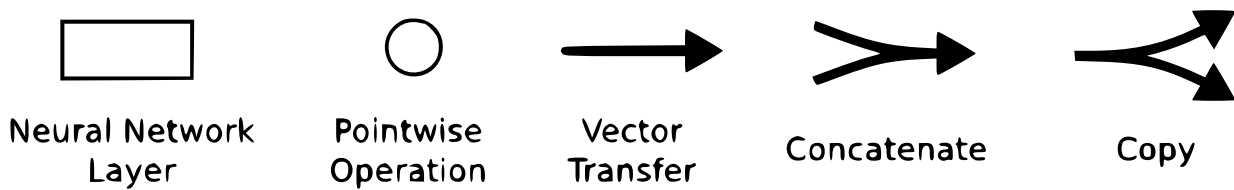


Figure 4.3: Diagram legend.

A side by side description of each layer of an LSTM and it's mathematical formulation can be seen in the following figures [20], where the dot, i.e.($X.W$), represents matrix multiplication.

The first layer represents the forget gate. This is where a decision is made on which information to keep and which to discard. The layer outputs a value between zero and one for each input because of the sigmoid function (represented as sigma in the equations below), meaning that a value of zero represents data that will be complicity erased and so forth, while an absolute value of one represents data that will be left unchanged for the next iteration.

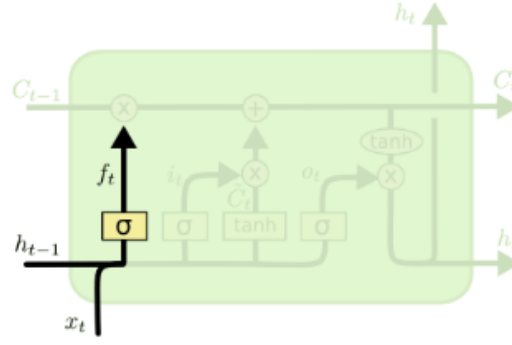


Figure 4.4: Forget Gate for concatenated inputs.

$$f = \sigma(X.W_f + b_f) \quad (4.1)$$

Similar to the forget gate, the LSTM structure also uses an update gate, to add new information to the system. As seen in Figure 4.5, the update gate generates the new values that will be used. This takes place by multiplying the update gate to the input layer, represented by X' where a hyperbolic tangent is used as the activation function. This is given as an example, as the Rectified Linear Unit (ReLU) or the softmax activation work perfectly fine as well. From the above-mentioned gates, a new internal state C can be computed as:

$$C_t = f * C_{t-1} + u * X' \quad (4.2)$$

Where the current state (C_t) is determined by what was forgotten (f), multiplied by the previous state (C_{t-1}) and added together with the update gate (u) times the input layer (X').

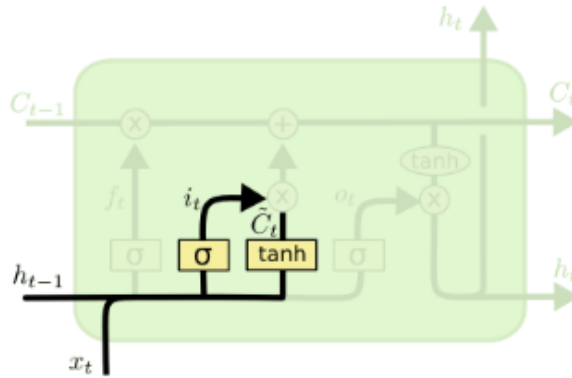


Figure 4.5: Update gate with the current state description.

$$U = \sigma(X.W_u + b_u) \quad (4.3)$$

$$X' = \tanh(X.W_c + b_c) \quad (4.4)$$

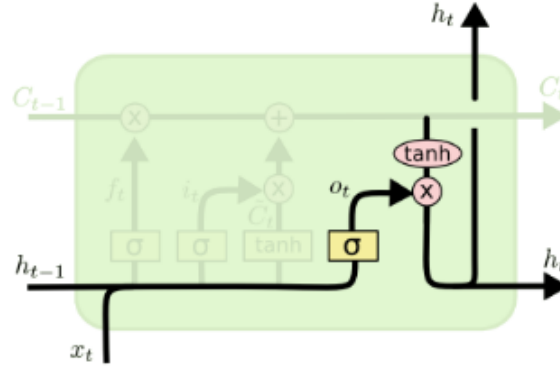


Figure 4.6: Result gate added to the output function.

$$r = \sigma(X.W_r + b_r) \quad (4.5)$$

The third and last gate is the result gate (r), which uses the previous internal state of the LSTM to generate a new internal state. As such, the output of the neural network can be written as:

$$Y_t = \text{softmax}(H_t.W + b) \quad (4.6)$$

4.3.4 Bi-directional Neural Networks

Bi-directional Recurrent Neural Networks (BiRNN) are used to increase the input information available to the network. As data flows in one direction, standard neural networks have inherited restrictions where the future input cannot be accessed by the current state [21]. Conversely, BiRNNs do not require their input data to be fixed. Furthermore, their future input information is available at the current state. This is done by connecting two hidden layers of opposite directions to the same output.

For an input sequence like: $X = \{x_1, x_2, x_3, \dots, x_n\}$, a neural network computes the hidden vector layer $h = \{h_1, h_2, h_3, \dots, h_n\}$ and output vector $y = \{y_1, y_2, y_3, \dots, y_n\}$ by iterating over $1 < t < n$ to generate the following equations:

$$h_t = \sigma(W_{xh} * x_t + W_{hh} * h_{t-1} + b_h) \quad (4.7)$$

$$y_t = W_{hy} * h_t + b_y \quad (4.8)$$

These equations expand on Section 4.2 and show how the neurons output a weighted sum of all the inputs and biases. Because of the promising results given by simple

LSTM networks, combining them with the benefits given by BiRNN, the gates showcased in Subsection 4.3.3 become:

$$i_t = \sigma(W_{xy} * x_t + W_{hi} * h_{t-1} + W_{ci} * c_{t-1} + b_i) \quad (4.9)$$

$$f_t = \sigma(W_{xf} * x_t + W_{hf} * h_{t-1} + W_{cf} * c_{t-1} + b_f) \quad (4.10)$$

$$c_t = f_t * c_{t-1} + i_t * \tanh(W_{xc} * x_t + W_{hc} * h_{t-1} + b_c) \quad (4.11)$$

$$o_t = \sigma(W_{xo} * x_t + W_{ho} * h_{t-1} + W_{co} * c_t + b_o) \quad (4.12)$$

$$h_t = o_t * \tanh(c_t) \quad (4.13)$$

Where i , f , o and c are the input gate, forget gate, output gate and cell activation vector. Because speech is a continuous action and entire utterances happen at once, simple RNNs fall behind in performance by only accessing previous information. Bidirectional RNNs process data in two directions, with one layer iterating forward (from $t = 1$ to n) and one layer iterating backwards (from $t = n$ to 1), seen in Figure 4.7 [22]. As such, the forward layer is described in Equation 4.14 while the backwards layer is described in Equation 4.15 and the updated output layer, Equation 4.16, becomes:

$$\vec{h}_t = \sigma(W_{x\vec{h}} * x_t + W_{\vec{h}\vec{h}} * \vec{h}_{t-1} + b_{\vec{h}}) \quad (4.14)$$

$$\overleftarrow{h}_t = \sigma(W_{x\overleftarrow{h}} * x_t + W_{\overleftarrow{h}\overleftarrow{h}} * \overleftarrow{h}_{t-1} + b_{\overleftarrow{h}}) \quad (4.15)$$

$$y_t = W_{\vec{h}y} * \vec{h}_t + W_{\overleftarrow{h}y} * \overleftarrow{h}_t + b_y \quad (4.16)$$

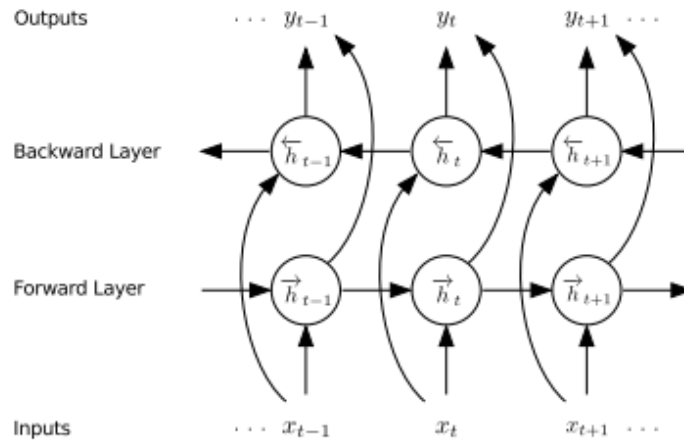


Figure 4.7: Visual representation of the hidden layers of one BiRNN

4.4 Types of Regulation Techniques

4.4.1 Dropout

Dropout is a regularization technique that involves shooting the neurons. It is applied after the activation function of each layer with the notable exception of the input and output layer. A percentage is used to determine the number of neurons that will remain in the networks. For example, a .75 pkeep value in the dropout function means that only 75 percent of the neurons will remain active during the training phase.

For the testing phase, all the neurons have to be present, meaning that a value of one has to be given to the dropout function to ensure a high accuracy rating. This helps the network understand the information it is given as it cannot presume that other neurons in the same layer have been activated and it prevents the network from just memorizing the values that were given as inputs.

The snippet of code below shows how dropout is used in TensorFlow for a layer of a neural network, where "pkeep" is the percentage of the neurons that will remain active and the call for the dropout is made after the activation function, in this case, the ReLU.

```
1 pkeep = tf.placeholder(tf.float32)
2 Yf = tf.nn.relu(tf.matmul(X, W) + b)
3 Y = tf.nn.dropout(Yf, pkeep)
```

Listing 4.1: Dropout for single network layer.

For a visual representation of a dropout rate given to a neural network, it can be seen Figure 4.8. A dropout rate of 40% given to the neural network on the left-hand side, which can be seen in the neural network on the right-hand side.

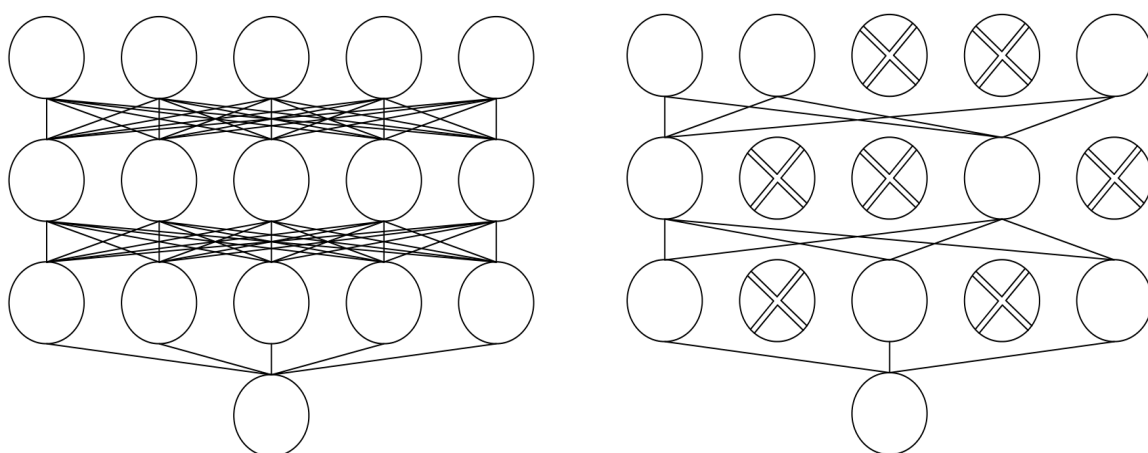


Figure 4.8: A 0.4 dropout rate on a four layer network.

4.4.2 Batch Normalisation

Statistics can be computed for each batch that is fed to the neural network so the average and the standard deviation can be computed. After computing them, subtracting the average and dividing by the standard deviation rescales and re-enters the output. In the equation, epsilon is added to avoid numerical problems as dividing by zero.

$$X = \frac{x - avg(x)}{stdev(x) + \epsilon} \quad (4.17)$$

The re-center and rescaling of the values can cause problems to the network for cases where data needs to be squeezed over to one side of the spectrum. To solve those cases, two degrees of freedom are added, α and β , for each neuron. By assigning the α value to the $stdev(x)$ and the β to $-avg(x)$ there will always be a case where the batch normalized value of x can be reverted to the original value [23].

$$BN(x) = \alpha * X + \beta \quad (4.18)$$

Batch normalization (BN) is added as a layer before the activation function because it makes use of the current weights and biases to compute the average and standard deviation. Because of the new values added by BN, biases are no longer useful. The β value can take the value of the bias if no better values were found. In contrast to the ReLU, when using a sigmoid, the scale parameter has to be used to make sure that the data fits in the useful part of the activation function. A representation of how to choose the right coefficients is given in Table 4.1.

Table 4.1: Parameters for different activation functions.

	ReLU	Sigmoid
Without BN	b	b
With BN	β	α, β

Chapter 5

Machine Learning Platform

5.1 TensorFlow

TensorFlow (TF) provides multiple Application Programming Interfaces (APIs) for machine learning. The lowest level API "TensorFlow Core" provides complete programming control [1]. For those who require fine levels of control over their models, TensorFlow Core is a well-suited tool for the job. There are higher level APIs that are built on top of TensorFlow Core. These higher level APIs are typically easier to learn and use than Tensorflow Core [1]. In addition, the higher level APIs such as "tf.estimator" helps with managing data sets, estimators, training and inferences (testing a trained network), as well as making repetitive tasks easier and more consistent [1].

The subsection below will start with TensorFlow Core. In order to gain an understanding of the basic principles that TensorFlow has to offer, a model shall be made. In the subsection after that, the same model will be implemented with tf.estimator. Knowing TensorFlow Core principles will give us a great mental model of how things are working internally when the more compact higher level API is used.

5.1.1 Tensors

The central unit of data in TensorFlow is the tensor. A tensor consists of a set of primitive values shaped into an array of any number of dimensions. A tensor's rank is its number of dimensions. Here are some examples of tensors:

```
1 3 # a rank 0 tensor; this is a scalar with shape []
2 [1., 2., 3.] # a rank 1 tensor; this is a vector with shape [3]
3 [[1., 2., 3.], [4., 5., 6.]] # a rank 2 tensor; a matrix with shape [2, 3]
4 [[[1., 2., 3.], [7., 8., 9.]]] # a rank 3 tensor with shape [2, 1, 3]
```

Listing 5.1: Tensor examples.

5.1.2 TensorFlow Core

Getting started with TensorFlow is done by using the import statement. This gives Python access to all of TensorFlow's classes, methods, and symbols. It is called by the following statement:

```
1 import tensorflow as tf
```

Listing 5.2: Importing Tensorflow library.

All the programs written with the help of TensorFlow core are built around computational graphs. They are a series of operations arranged into a graph and connected by nodes. To see the outcome of such a program, firstly the computational graph needs to be created and after that, it needs to be run. This provides a contrast to normal code written in python and to display this, the print function for a tensorflow constant is called bellow.

```
1 node1 = tf.constant(3.0, dtype=tf.float32)
2 node2 = tf.constant(4.0) # also tf.float32 implicitly
3 print(node1, node2)
4
5 Output:
6 Tensor("Const:0", shape=(), dtype=float32)
7 Tensor("Const_1:0", shape=(), dtype=float32)
```

Listing 5.3: Printing output without sess.run().

As seen in Listing 5.3, printing an element does not show the value that it holds and it rather shows the technical details behind, such as the element being a constant of type float32. To investigate the contents of any element in TF, an object of type session needs to be defined. When the new object is run, a new session is generated and the content of the element can be seen. All code written in tensorflow core follows this basic principle.

```
1 sess = tf.Session()
2 print(sess.run([node1, node2]))
3
4 Output:
5 [3.0]
```

Listing 5.4: Printing output with sess.run().

Other basic elements in tensorflow are placeholders (tf.placeholder), they can be changed to accept external inputs and variables. Variables allow the system to change its outputs while keeping the same inputs, this allows the model to be trainable.

5.1.3 tf.train API

The tf.train API holds optimizers that change each variable to minimize the loss function. One of the simplest optimizers used to train neural networks is gradient descent.

Each variable is modified by the magnitude of the derivative of loss with respect to that variable. Using a computer to gain these gradients is far less prone to error than doing it by hand. Each of the gradients will point towards the minimum of the loss function and when solving for the gradient a step needs to be determined and used as the rate of change. It is important to have a small step as to not jump over the valley where the function takes its minimum value, but in the beginning to reduce time and processing power a bigger step could be used. One solution to this problem is to use a variable step input that changes as the gradients are determined. TensorFlow can automatically produce derivatives given only a description of the model using the function `tf.gradients`. For simplicity, optimizers typically do this for us.

```
1 optimizer = tf.train.GradientDescentOptimizer(0.01)
2 train = optimizer.minimize(loss)
3 sess.run(train)
```

Listing 5.5: The gradient descent optimizer.

The gradient descent optimizer takes an input, defined as 0.01 in the example shown in Listing 5.5, that represents the step input or the rate of change for the gradients.

5.1.4 `tf.estimator` API

To simplify the procedure of machine learning, `tf.estimator` can be used as a higher-level library within Tensorflow. It helps the user with running, training and evaluation loops, managing data sets and much more. Making the entire process of writing an algorithm and maintaining it be less tedious. Although the estimator library has a set of predefined models to make things easier, a custom model can be created while keeping the high level abstraction of the data set, training, and feeding. A linear regression algorithm built with `tf.estimator` is included below to show how the library works [24]:

```

1 import numpy as np
2 import tensorflow as tf
3
4 # Declare list of features. We only have one numeric feature. There are many
5 # other types of columns that are more complicated and useful.
6 feature_columns = [tf.feature_column.numeric_column("x", shape=[1])]
7
8 # An estimator is the front end to invoke training ( fitting ) and evaluation
9 # (inference). There are many predefined types like linear regression,
10 # linear classification , and many neural network classifiers and regressors.
11
12 # The following code provides an estimator that does linear regression.
13 estimator = tf.estimator.LinearRegressor(feature_columns=feature_columns)
14
15 # TensorFlow provides many helper methods to read and set up data sets.
16 # Here we use two data sets: one for training and one for evaluation
17 # We have to tell the function how many batches
18 # of data (num_epochs) we want and how big each batch should be.
19 x_train = np.array ([1., 2., 3., 4.])
20 y_train = np.array ([0., -1., -2., -3.])
21 x_eval = np.array ([2., 5., 8., 1.])
22 y_eval = np.array ([-1.01, -4.1, -7, 0.])
23 input_fn = tf.estimator.inputs.numpy_input_fn(
24     {"x": x_train}, y_train, batch_size=4, num_epochs=None, shuffle=True)
25 train_input_fn = tf.estimator.inputs.numpy_input_fn(
26     {"x": x_train}, y_train, batch_size=4, num_epochs=1000, shuffle=False)
27 eval_input_fn = tf.estimator.inputs.numpy_input_fn(
28     {"x": x_eval}, y_eval, batch_size=4, num_epochs=1000, shuffle=False)
29
30 # We can invoke 1000 training steps by invoking the method and passing the
31 # training data set.
32 estimator.train(input_fn=input_fn, steps=1000)
33
34 # Here we evaluate how well our model did.
35 train_metrics = estimator.evaluate(input_fn=train_input_fn)
36 eval_metrics = estimator.evaluate(input_fn=eval_input_fn)
37 print("train metrics: %r"% train_metrics)
38 print("eval metrics: %r"% eval_metrics)

```

Listing 5.6: Linear regression algorithm built with `tf.estimator`.

5.2 Batch Size, Dropout and Learning Rate Comparison

In order to gain a better understanding on how the different parameter values (batch size, dropout and learning rate) may affect the performance (training duration and test accuracy) of a neural network model, a small experiment was conducted.

This experiment consists of six test trials. For each test trial different parameter values were chosen, as shown in table 5.1.

Table 5.1: Six tests with different Parameter values.

Parameters	Test1 -●-	Test2 -●-	Test3 -●-	Test4 -●-	Test5 -●-	Test6 -●-
Batch Size	2 2 2	20 20 20	20 20 20	20 20 20	50 20 20	50 20 20
Dropout	0.05	0.05	0.00	0.50	0.05	0.05
Learning Rate	0.001	0.001	0.001	0.001	0.001	0.010

Batch Size: (train data, evaluation data, test data)

Note that the batch size is represented by the training, evaluation and testing data and will be represented this way for all the following tables.

The tests were run on a model designed by the Silicon Valley Data Science company, which is based on the TensorFlow implementation of Baidu's DeepSpeech architecture on the Mozilla' DeepSpeech repository. The GitHub repositories (silicon-valley-data-science and Mozilla) can be found respectively in the following citations: [10, 25]. The model consists of a Bi-Directional Recurrent Neural Network with three fully connected layers before it and a fully connected layer with an activation layer after it.

The data used in the model referred above was a small set of WAV files consisting of recorded audio of the numbers:

{zero, one, two, three, four, five, six, seven, eight, nine},

and their respective labels in text format. The data is separated into folders:

- Train: train-clean-100-wav
- Test: test-clean-wav
- Dev: dev-clean-wav

In this section, the train and test results given in the following graphs and tables will be referred. However, for the interested reader, all three train, test, and evaluation results have been included in appendix B with slightly scaled up graphs for a more detailed analysis. Here the goal is to visualize and compare the different graph curves and final results shown in the respective tables. For consistency, all the tests are run for 25 Epochs.

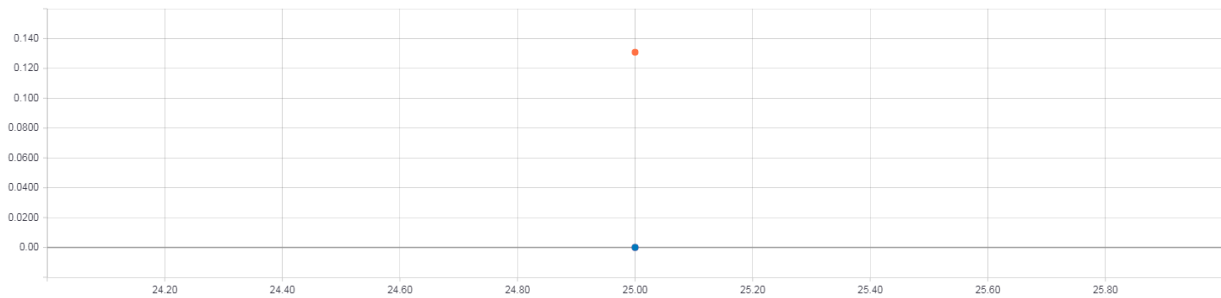
5.2.1 Batch Size Test

In this subsection three test from Table 5.1 will be used for the batch size test has shown in Table 5.2. For each trial test only the batch size parameter has been changed.

Table 5.2: Three tests with different batch size values.

Parameters	Test1 -●-	Test2 -●-	Test5 -●-
Batch Size	2 2 2	20 20 20	50 20 20
Dropout	0.05	0.05	0.05
Learning Rate	0.001	0.001	0.001

The following graph (Fig. 5.1) and table (Tab. 5.3) represent the error rate for the test data set (the one that is used to test the models accuracy after training). The results show that Test1 has $\sim 13\%$ word error rate (WER) while Test2 and Test3 is $\sim 0\%$ WER.

**Figure 5.1:** Test error rate.**Table 5.3:** Test error rate results.

Tests	Value	Epoch	Duration
Test1 -●-	0.1308	25.00	0s
Test2 -●-	0.000	25.00	0s
Test5 -●-	0.000	25.00	0s

The following graph (Fig. 5.2) and table (Tab. 5.4) represent the error rate for the train data set (the one that is used to train the model). The results show that Test1 took a duration of $\sim 1h26m13s$ for the training while Test2 and Test3 is $\sim 18m50s$ and $\sim 14m50s$ respectively.

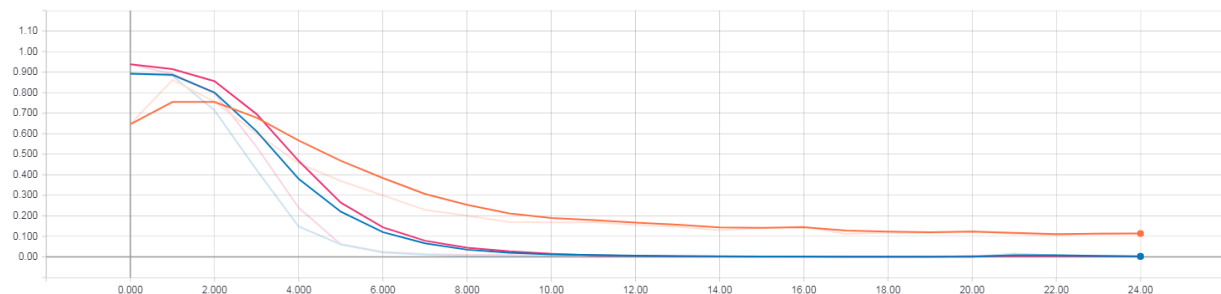
**Figure 5.2:** Training error rate.

Table 5.4: Training error rate results.

Tests	Value	Epoch	Duration
Test1 -●-	0.1148	24.00	1h 26m 13s
Test2 -●-	9.5073e-4	24.00	18m 50s
Test5 -●-	5.1858e-4	24.00	14m 50s

The final results show that Test1 with the smaller batch size took significantly longer to train and had the worst accuracy. However, Test3 with the larger batch size took the least amount of time and had the same accuracy as Test2.

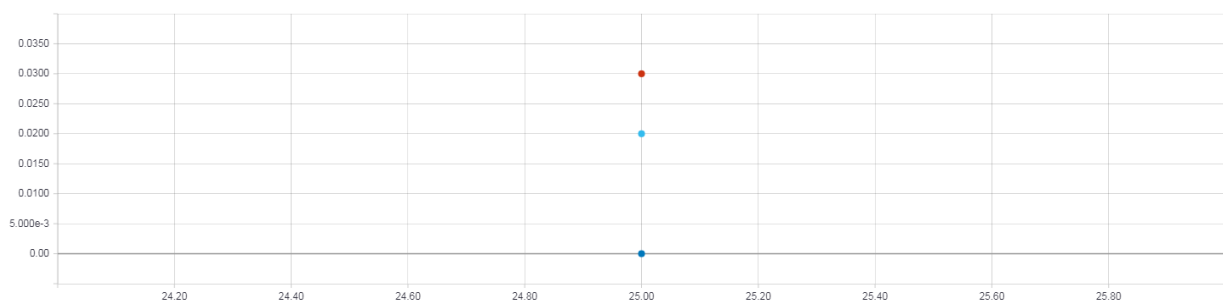
5.2.2 Dropout Test

In this subsection three test from Table 5.1 will be used for the batch size test as shown in Table 5.5. For each trial test, only the dropout parameter has been changed.

Table 5.5: Three tests with different dropout values.

Parameters	Test2 -●-	Test3 -●-	Test4 -●-
Batch Size	20 20 20	20 20 20	20 20 20
Dropout	0.05	0.00	0.50
Learning Rate	0.001	0.001	0.001

The following graph (Fig. 5.3) and table (Tab. 5.6) represent the error rate for the test data set. The results show that Test2 has $\sim 0\%$ word error rate (WER) while Test3 and Test4 is $\sim 3\%$ and $\sim 2\%$ WER respectively.

**Figure 5.3:** Test error rate.**Table 5.6:** Test error rate results.

Tests	Value	Epoch	Duration
Test2 -●-	0.000	25.00	0s
Test3 -●-	0.030	25.00	0s
Test4 -●-	0.020	25.00	0s

The following graph (Fig. 5.4) and table (Tab. 5.7) represent the error rate for the train data set. The results show that Test 2 took a duration of $\sim 18m50s$ while Test2 and Test3 took $\sim 18m5s$ and $\sim 18m10s$ respectively.

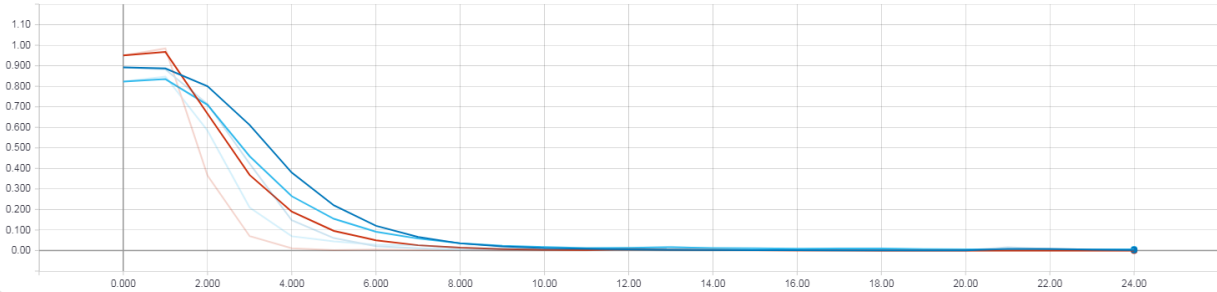


Figure 5.4: Training error rate.

Table 5.7: Training error rate results.

Tests	Value	Epoch	Duration
Test2 -●-	9.5973e-4	24.00	18m 50s
Test3 -●-	0.000	24.00	18m 5s
Test4 -●-	6.3331e-3	24.00	18m 10s

The final results show that Test3 with no dropout and Test4 with a high dropout of 50% had a slightly faster training time, However, the test error rate was slightly worse than Test2 with a dropout of 5%. Overall all three tests performed very well.

5.2.3 Learning Rate Test

In this subsection two test from Table 5.1 will be used for the batch size test as shown in Table 5.8. For each trial test, only the learning rate parameter has been changed.

Table 5.8: Two tests with different learning rate values.

Parameters	Test5 -●-	Test6 -●-
Batch Size	50 20 20	50 20 20
Dropout	0.05	0.05
Learning Rate	0.001	0.010

The following graph (Fig. 5.5) and table (Tab. 5.9) represent the error rate for the test data set. The results show that Test5 has $\sim 0\%$ word error rate (WER) while Test6 is $\sim 89\%$ WER.

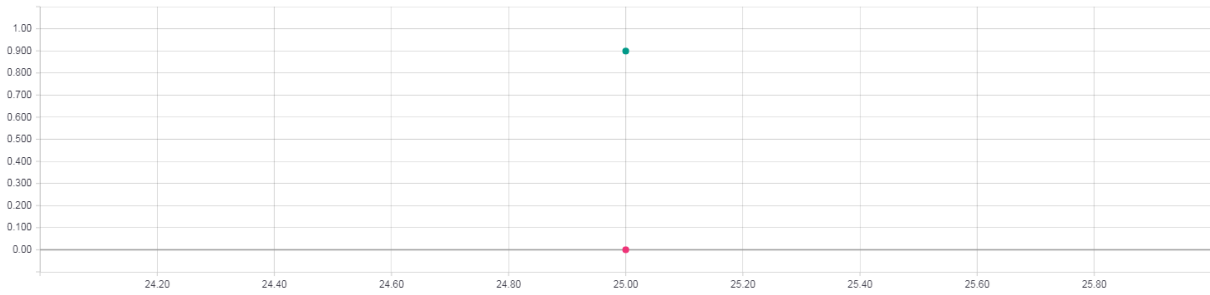


Figure 5.5: Test error rate.

Table 5.9: Test error rate results.

Tests	Value	Epoch	Duration
Test5 -●-	0.000	25.00	0s
Test6 -●-	0.8992	25.00	0s

The following graph (Figure 5.6) and Table 5.10 represent the error rate for the train data set. The results show that Test5 took a duration of $\sim 14m50s$ while Test6 took $\sim 12m29s$.

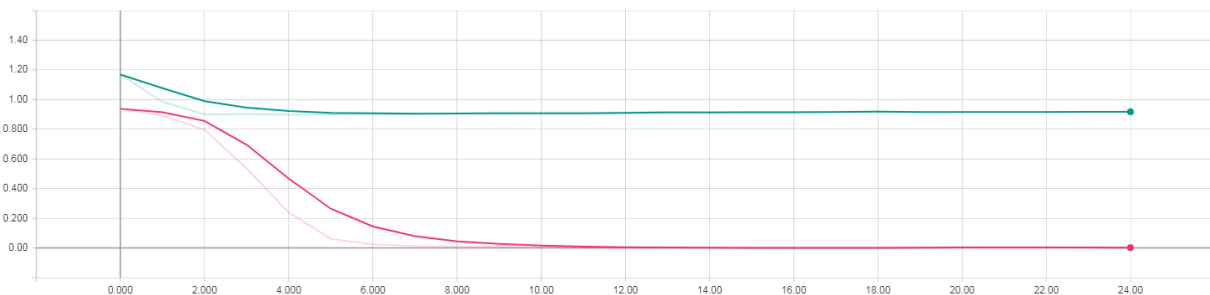


Figure 5.6: Training error rate.

Table 5.10: Training error rate results.

Tests	Value	Epoch	Duration
Test5 -●-	5.1858e-4	24.00	14m 50s
Test6 -●-	0.9188	24.00	12m 29s

The final results show that Test5 with learning rate of 0.1% outperformed Test6 with a learning rate of 1% by a long shot. The graph clearly shows the model didn't converge to zero at all, unlike Test5, meaning it didn't learn anything during training.

In conclusion, there is no clear way to set the parameters, why this particular learning rate, how long to we leave it training, what is a good dropout value, etc. It all depends

on the model you are training, and even then there is no real explanation of what is or isn't a good parameter value. There is no methodical way of going about it, but when the parameters are set right it works far better than the traditional methods.

Chapter 6

Model Development

One of the goals of this project was to research, design and compare different neural network topologies. Some of these were feed-forward and recurrent. Continuing, some neural networks were made deep, first by adding fully connected layers to the existing type feed-forward or RNN. Different configurations were tested, some proving to be effective for achieving designated goals, and some less so. Subject to further research, we found that recurrent neural networks work best for speech recognition. Therefore the following comparison will be based on different RNNs configurations.

6.1 Neural Network Comparison

In this chapter two models are developed based on the simple LSTM model from the silicon-valley-data-science RNN tutorial [10]. Firstly these models (including "their simple LSTM model") will be trained and tested on a small dataset with the numbers:

{zero, one, two, three, four, five, six, seven, eight, nine},

the same data used in Chapter 4 for the parameter comparison. After that, the best performing model shall be picked for a more detailed description (with code) and further training with a bigger dataset of the English language.

Before the comparison of the 3 models, let's take a look at their structure in the following diagrams. As mentioned earlier the two models that were developed are based on the silicon-valley-data-science simple LSTM model. Their model consists of two LSTM layers as shown in the diagram below (Figure 6.1).

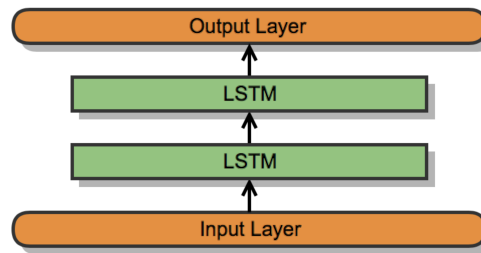


Figure 6.1: Simple LSTM model.

Based on the model above, it was decided among us to make an addition of two fully connected (FC) layers before and one FC layer after the two LSTM layers (shown in Figure 6.2), with the desire of gaining better accuracy.

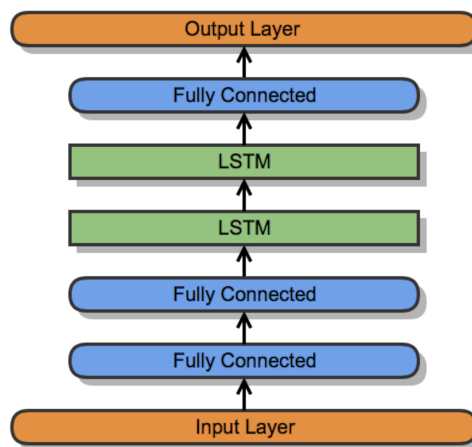


Figure 6.2: Simple LSTM model with fully connected layers.

After that, a decision was made to replace the two LSTM layers in the model above with one Bi-Directional LSTM layers (seen in Figure 6.3), again with the desire to make a more accurate model

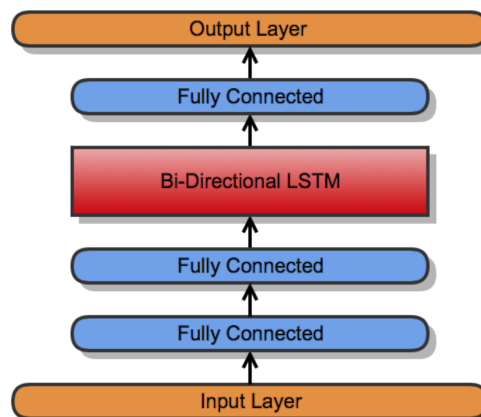


Figure 6.3: BiRNN model with fully connected layers.

In order to have a fair comparison between these model the parameters values were made to be the same for all models as shown in Table 6.1. The respective models are mapped to the described ones in Table 6.2. For the interested reader an expanded version in appendix C.

Table 6.1: Parameter values of the three model.

Parameters	Model1 -●-	Model2 -●-	Model3 -●-
Batch Size	50 20 20	50 20 20	50 20 20
Dropout	0.05	0.05	0.05
Learning Rate	0.001	0.001	0.001

Table 6.2: Models.

Model1 -●- (simple LSTM Model)
 Model2 -●- (simple LSTM + FC Model)
 Model3 -●- (BiRNN + FC Model)

The following graph (Fig. 6.4) and table (Tab. 6.3) represent the error rate for the test data set. The results show that Model1 has $\sim 20\%$ word error rate (WER) while Model2 and Model3 is $\sim 8\%$ and $\sim 0\%$ WER respectively.

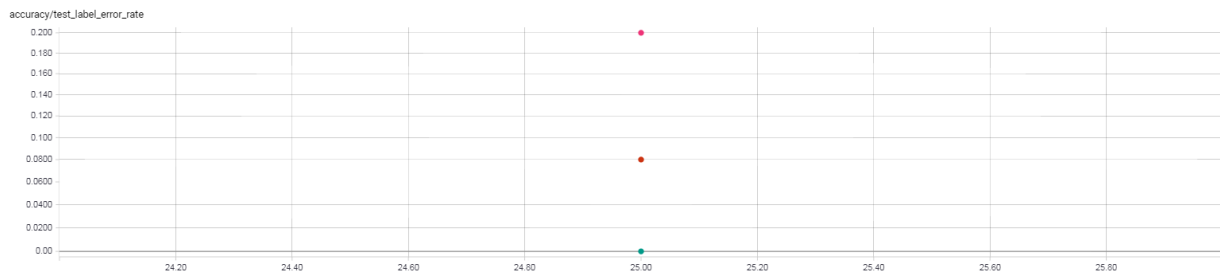


Figure 6.4: Test error rate.

Table 6.3: Test error rate results.

Models	Value	Epoch	Duration
Model1 -●-	0.200	25.00	0s
Model2 -●-	0.080	25.00	0s
Model3 -●-	0.000	25.00	0s

The following graph (Fig. 6.5) and table (Tab. 6.4) represent the error rate for the validation data set. The results show that Model1 has took $\sim 7m9s$ to complete training while Model2 and Model3 is $\sim 9m8s$ and $\sim 11m49s$ respectively.

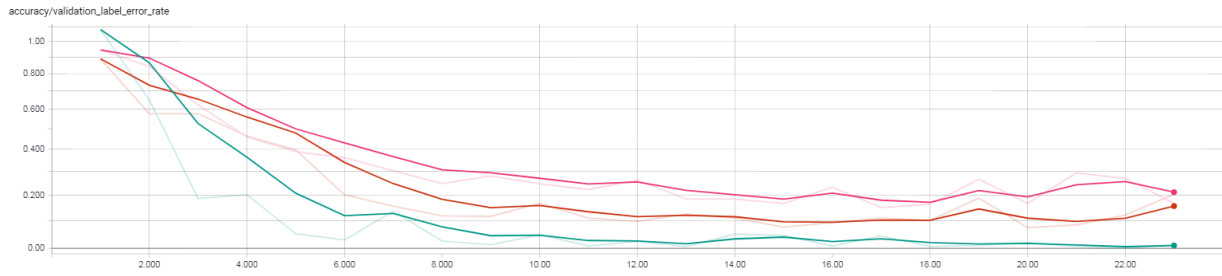


Figure 6.5: Validation error rate.

Table 6.4: Validation error rate results.

Models	Value	Epoch	Duration
Model1 -●-	0.1669	23.00	7m 9s
Model2 -●-	0.2030	23.00	9m 8s
Model3 -●-	0.01398	23.00	11m 49s

The final results show that Model1 was improved by adding the fully connected layers (Model2) from $\sim 20\%$ to $\sim 8\%$ WER and further improved by the replacing the two LSTMs by one Bi-Directional LSTM layer (Model3) from $\sim 8\%$ to $\sim 0\%$ WER. The graph for the validation error rate (Figure 6.5) clearly shows the three distinct lines in "parallel" with each other. The top line being the least accurate and the bottom one being the most.

6.2 Model Description

As shown in Chapter 6 under Neural Network Comparison, the model that is used in this DSR system is based on a bidirectional, Long-short Term Memory, recurrent neural network. This recurrent network is using the standard TensorFlow implementation of an LSTM cell, called "BasicLSTMCell", and used as a bidirectional layer. There are two cells in the layer, one "forward" direction and one "backward" direction. Each cell is unrolled 1024 times, giving the layer 2048 elements. The purpose is to recognize the context of the text, thus improving the prediction accuracy. The complete topology for the network is as follows (an expanded version of Figure 6.6 can be found in Appendix B):

The second layer has much of the same functionality as the first. However, they differ in the output dimension size, because the output of this layer is going to be the input for the bi-directional LSTM layer.

```

1  # 2nd layer
2  with tf.name_scope('fc2'):
3      b2 = variable_on_cpu('b2', [n_hidden_3], tf.random_normal_initializer(stddev=b2_stddev))
4      h2 = variable_on_cpu('h2', [n_hidden_1, n_hidden_3], tf.random_normal_initializer(stddev=
5      h2_stddev))
6      layer_2 = tf.nn.minimum(tf.nn.relu(tf.add(tf.matmul(layer_1, h2), b2)), relu_clip)
7      layer_2 = tf.nn.dropout(layer_2, (1.0 - dropout[1]))
8
9      tf.summary.histogram("weights", h2)
10     tf.summary.histogram("biases", b2)
11     tf.summary.histogram("activations", layer_2)

```

Listing 6.2: Second fully connected layer

The next layer defines two LSTM cells. Both are initialized with the default forget bias, in order to remember more information at the beginning of training. For these cells, there is no dropout.

```

1  with tf.name_scope('lstm'):
2      # Forward direction cell:
3      lstm_fw_cell = tf.contrib.rnn.BasicLSTMCell(n_cell_dim, forget_bias=1.0, state_is_tuple=True)
4      lstm_fw_cell = tf.contrib.rnn.DropoutWrapper(lstm_fw_cell,
5      input_keep_prob=1.0 - dropout[3],
6      output_keep_prob=1.0 - dropout[3])
7
8      # Backward direction cell:
9      lstm_bw_cell = tf.contrib.rnn.BasicLSTMCell(n_cell_dim, forget_bias=1.0, state_is_tuple=True)
10     lstm_bw_cell = tf.contrib.rnn.DropoutWrapper(lstm_bw_cell,
11     input_keep_prob=1.0 - dropout[4],
12     output_keep_prob=1.0 - dropout[4])

```

Listing 6.3: BiRNN Layer

Further on, the input is reshaped into a three-dimensional tensor, to be compatible with the BiRNN layer, and the network is unrolled.

```

1  outputs, output_states = tf.nn.bidirectional_dynamic_rnn(cell_fw=lstm_fw_cell,
2  cell_bw=lstm_bw_cell,
3  inputs=layer_2,
4  dtype=tf.float32,
5  time_major=True,
6  sequence_length=seq_length)
7
8  tf.summary.histogram("activations", outputs)

```

Listing 6.4: BiRNN Layer

Following, the previous outputs is reshaped again into a two-dimensional tensor and sent to the next fully connected layer. This layer is similar to the first two, with the main

operation being tensor multiplication and addition. Afterwards, ReLU and drop-out is applied.

```

1  with tf.name_scope('fc4'):
2      # Now we feed 'outputs' to the fifth hidden layer with clipped RELU activation and dropout
3      b5 = variable_on_cpu('b5', [n_hidden_5], tf.random_normal_initializer(stddev=b5_stddev))
4      h5 = variable_on_cpu('h5', [(2 * n_cell_dim), n_hidden_5], tf.random_normal_initializer(
5          stddev=h5_stddev))
6      layer_5 = tf.nn.minimum(tf.nn.relu(tf.add(tf.matmul(outputs, h5), b5)), relu_clip)
7      layer_5 = tf.nn.dropout(layer_5, (1.0 - dropout[5]))
8
9      tf.summary.histogram("weights", h5)
10     tf.summary.histogram("biases", b5)
11     tf.summary.histogram("activations", layer_5)

```

Listing 6.5: Third fully connected layer

The output layer, is used as the end point of the network. Multiplication and addition operations are applied to the previous output tensor. At this point, the output was transformed by the entire network. The last step is to reshape the tensor back into a 3D form, after which it is passed to another function for decoding.

```

1  with tf.name_scope('fc5'):
2      # Now we apply the weight matrix 'h6' and bias 'b6' to the output of
3      # 'layer_5'
4      # creating 'n_classes' dimensional vectors, the logits.
5      b6 = variable_on_cpu('b6', [n_hidden_6], tf.
6          random_normal_initializer(stddev=b6_stddev))
7      h6 = variable_on_cpu('h6', [n_hidden_5, n_hidden_6], tf.
8          random_normal_initializer(stddev=h6_stddev))
9      layer_6 = tf.add(tf.matmul(layer_5, h6), b6)
10
11     tf.summary.histogram("weights", h6)
12     tf.summary.histogram("biases", b6)
13     tf.summary.histogram("activations", layer_6)

```

Listing 6.6: Output layer

Chapter 7

Network Framework

Completing the second part of the project requires implementing the Speech Recogniser in a distributed manner. High level, the system requires open access over a network, local or otherwise. Low level, the recognizer must be able to communicate over the available network. In order to demonstrate the system's complete functionality, a Virtual Private Network (VPN) is used to provide open access between devices and sockets are used to send data through the network.

7.1 Virtual Private Network (VPN)

The VPN extends a private network over a public network and enables devices to send and receive data across shared or public networks, in the same way as if they would be directly connected to the same network. Due to the fact that multiple devices are in various locations and on different networks, some of which have security protocols enabled, direct communication between these devices is not possible without a VPN.

OpenVPN is the open-source software application that was utilized here. It implements VPN techniques for creating secure point-to-point connections in routed or bridged configurations and remote access. A custom security protocol, based on SSL/TLS is enforced on the network, and it is capable of traversing network address translators (NATs) and firewalls.

For the current project, OpenVPN was used in a multiclient-server configuration. The server itself was installed on a Raspberry Pi 3. Authentication certificates were released for every client that required access to the network [26]. The network setup is shown in Figure 7.1.

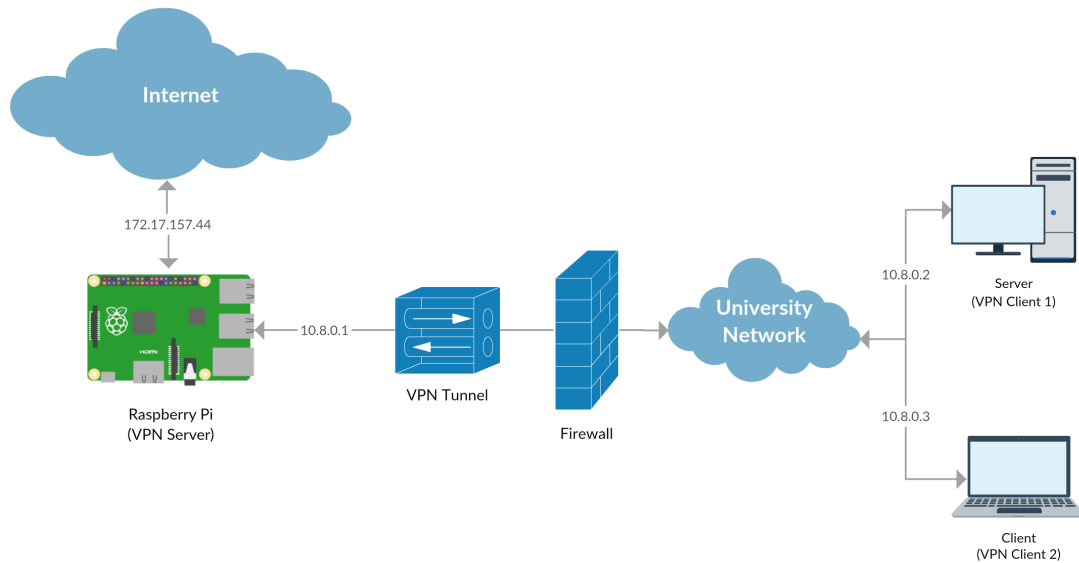


Figure 7.1: Virtual private network to university.

7.2 Sockets

Network sockets are software interfaces that allow communication between two different processes on the same or on different machines. Most often utilized in client-server frameworks, a network socket represents one endpoint in a communication flow between two programs.

In this project, TCP sockets were implemented in Python to provide support for a variety of desktop-based operating systems. When the server is initialized, it creates a socket and listens for incoming requests.

```

1 (HOST,PORT) = (server_ip,port)
2 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
3 s.bind((HOST, PORT))
4 s.listen (numb_of_clients)
5 return s

```

Listing 7.1: Server initialization.

On the client side, another TCP socket is created and using the server's IP address and port number, a connection is established.

```

1 (HOST,PORT)=('10.8.0.2',2001)
2 s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);
3 s.connect((HOST,PORT))

```

Listing 7.2: Client initialization.

Following a successful connection, the client sends audio information to the server, which will return the decoded data in text form.

Chapter 8

Implementation

8.1 Data Mining

Data Set

Any neural network requires data in order to learn and become useful. As such, the training data is an important part of the learning process. For this implementation, the OpenSLR LibriSpeech ASR corpus was imported to provide training data.

The LibriSpeech is comprised of approximately 1000 hours of 16kHz English read speech in .flac format and was prepared by Vassil Panayotov with the assistance of Daniel Povey. It is based on audio books from the LibriVox project that have been segmented and aligned. It contains 3 different sets, differentiated by type and size.

The first, and smallest one is made of 6.3 GB of "clean" speech, representing approx. 100 hours. "Clean" speech refers to audio recordings where noise and disturbances that might corrupt their quality were eliminated as much as possible. The second set is made of 23 GB of "clean" speech that represents approx. 360 hours.

The final set is made of 30 GB of "other" speech, representing approx. 500 hours of audio. "Other" speech refers to recordings that contain a certain degree of audio disturbance. The corpus also contains smaller sets for validation and testing.

Data Processing

As presented above, the OpenSLR corpus provides many features for training neural networks. It is especially effective for speech recognition systems. However, the current implementation was designed to use pairs of **.wav** files and **.txt** files as input. Each **.wav** file represents the audio information that is encoded into the corresponding **.txt** file, more specifically short paragraphs.

Unfortunately, this data set is prepared in a different format. There is a single `.txt` file that corresponds to many `.wav` files and these are organized in bigger sets. Because of this, the data in its original form is not usable for the current neural network. To prepare the data, certain scripts were implemented in Python.

The first algorithm is designed to navigate a folder path and its sub-folders to convert all `.flac` files to `.wav`. It uses the `os` and `pydub` libraries. To access every sub-folder in the path, `os.walk` is used. This function returns the root folder and a list of files and directories in root, after which an internal folder becomes the new root

```

1 def folder_navigator_convert(source, dest):
2     """ Function that goes in a folder and all subfolders and converts .flac files into .wav files.
3         Parameters: 'source' – string, abs path to the root folder, 'dest' – string, abs path to dest
4             folder (both must be with a '/' at the end!) """
5
6     for root, dirs, files in os.walk(source):
7         for name in files:
8             abs_path = os.path.join(root, name)
9
10            if '\\\\' in abs_path:
11                abs_path = abs_path.replace("\\\\", '/')
12
13            if flac_check(name):
14                flac_to_wav(abs_path, name, dest)
15            else:
16                print(("File %s ignored!")%(name))

```

Listing 8.1: Folder navigator.

The conversion is handled by another function that uses the `pydub` library to load the `.flac`, parse the name and export the converted `.wav` file.

```

1 def flac_to_wav(input_flac, input_name, dest):
2     """ Function that converts a .flac to a .wav file. Parameter: 'input_flac' – string abs path to a
3         .flac,
4         'input_name' – string of only the name of the .flac (with format), 'dest' string abs path to
5         destination folder (with a '/' in the end!) """
6
7     audio_file = AudioSegment.from_file(input_flac)
8     temp_name = dest
9     temp_name += input_name
10    temp_name = temp_name[:len(temp_name)-4]
11    temp_name += ".wav"
12    audio_file.export(temp_name)
13    print(("File %s conv success!")%(input_name))

```

Listing 8.2: Flac to wav conversion.

The second algorithm navigates the folders in the same way, by using the `os.walk` method. Then it checks for `.txt` files by parsing their name.

```

1 def txt_check(input_name):
2     """ Function that checks if a file is .txt or not. Returns True or False (bool)
3         Parameter: 'input_name' string, that has the name of the file (with format). It may work with
4         absolute path and name (maybe) """
5
6     _, file_extension = os.path.splitext(input_name)
7
8     if file_extension == '.txt':
9         return True
10    else:
11        return False

```

Listing 8.3: Text file check.

The format of the text files in this audio set is known. Using this information, the file name and content is parsed, creating new text files for every paragraph that corresponds to a .flac file. This is achieved by exploiting the file name in relation to the beginning and end of each paragraph inside the file.

```

1 str_temp = fo.read()
2
3 char_to_look_for = " "
4 ctrl_seq_pos = str_temp.find(temp_name)
5 first_space = str_temp.find(char_to_look_for)
6
7 while ctrl_seq_pos != -1:
8
9     temp_start_content = first_space+1
10    new_file_name = dest
11    new_file_name += str_temp[ctrl_seq_pos:first_space] + ".txt"
12
13    ctrl_seq_pos = str_temp.find(temp_name,ctrl_seq_pos+20)
14
15    new_fo = open(new_file_name,"w")
16    new_file_counter+=1

```

Listing 8.4: Text file parsing.

8.2 DSR System Implementation

The complete DSR system is the sum of all the previously described parts. Firstly, a recurrent neural network model was designed and implemented. Following, the model was trained on the available data. Afterwards, the distributed part was focused on. The VPN was set up, and the client-server architecture took form using Python-based, TCP sockets.

Normal operation of the system is as follows:

1. The server is initialized
2. The client connects to the server
3. The client sends an audio file
4. The server decodes the audio into text
5. The server sends the text to the client

The file sent by the client can be either a microphone recording stored as a .wav at runtime or an existing .wav file. To store this file, the server expects to receive multiple chunks of a fixed byte size from the client, shown below:

```

1 with open(recv_file_name,'wb') as f:
2     while True:
3         l=connSock.recv(1024)
4         if not l:
5             break
6         f.write(l)

```

Listing 8.5: Server receiving file.

After a successful transfer, the server forks a new process. This new process runs a script that loads the trained neural network and decodes the received audio file. A pipe is opened between the two processes for the decoded text to be sent back to the client.

```

1 p = subprocess.Popen(['python', 'C:/Users/Mavericks/Documents/GitHub/Mavericks/coding/
2   modelLoadPyV2/modelLoadPyV2/modelLoadPyV2.py', train_option, model_name, 'S117'],
   stdin=subprocess.PIPE, stdout=subprocess.PIPE, encoding='utf8')

```

Listing 8.6: New process.

When this script is called, the neural network is initialized. First, the set of operations which will handle tensors, called the computational graph, is loaded. TensorFlow uses Python to build the "skeleton" of a neural network and uses it to connect to the C++ runtime that handles the operations when data is inserted. Second, the final numerical values for all trained variables are loaded into the computational graph. At this point the network is ready to inference.

```

1 self.saver.restore(self.sess, tf.train.latest_checkpoint(self.model_path))
2 logger.info(section.format('Model loaded successfully for predictions!'))
3 logger.info(section.format('Decoding client input data!'))

```

Listing 8.7: Model restore.

The decoded text is received by the client and displayed on screen.

8.3 Testing

The testing methodology was to simulate the real world application for the DSR system. In order to test the DSR system, the model was given different speech samples to decode into text. The nature of the tests implies that there are no labels for the system to calculate the error rate.

The following tests show the original speech and the respective decoded speech:

Test 1

Original speech: "he began a confused complaint against the wizard who had vanished behind the curtain on the left".

```
kamranthomas at kamy-mac in ~/Desktop/speech_recognizer/sockets
$ python3 client.py
Message sent to server!
Prediction of message: he began a confused complaint against the wizard who advanced behind the curtain on the left
```

Figure 8.1: Model prediction for first sample.

Test 2

Original speech: "now open your mouth and speak if your thoughts incline ever so little towards fuming you will say fuming furious if they turned by even a hairs breath towards furious you will say furious fuming but"

```
kamranthomas at h220 in ~/Desktop/speech_recognizer/sockets
$ python3 client.py
Message sent to server!
Prediction of message: now open your mouth and speak if your thoughts incline ever so little towards fuming you will
say fuming furious if they turned by even a hairs breath towards furious you will say furious fuming but
```

Figure 8.2: Model prediction for second sample.

Test 3

Original speech: "the side of the sleigh which was next his own house"

```
kamranthomas at h220 in ~/Desktop/speech_recognizer/sockets
$ python3 client.py
Message sent to server!
Prediction of message: the side of the sleigh which was next his own house
```

Figure 8.3: Model prediction for third sample.

Based on the results, even though there were some grammatical and contextual mistakes, the validity of the neural network is proven. This is especially relevant considering the hardware limitation and time constraints. The model only trained on the number set and on the 6.3 GB of "clean" speech, representing approx. 100 hours.

Chapter 9

Discussion

In chapter 2, specific success criteria were created to limit the scope and also allow for the project to be evaluated. This discussion will focus on what was accomplished in terms of success criteria and on the possible future steps to improve or advance this project. The list of success criteria from Section 2.2 is shown below:

- Data mining
- Choose a filter
- Design a deep neural network model
- Train for the English language
- Create a client-server framework
- Implement the distributed model

In order to use the OpenSLR training set, it was crucial to develop and use the scripts detailed in Section 8.1. Having an algorithm capable of following a path from a main folder and its sub-folders to convert all the necessary files to the right format meant that the amount of data increased drastically. Using up to 1000 hours of speech for the neural network to learn from was crucial in the development phase. As no neural network, no matter how complex and well thought out, could have a scope of understanding the full English language from a limited data set. Another important difference is to be made between clean speech and noisy speech, as a network trained only with clean speech stands no chance of recognizing our voices. The preprocessing part covered in Chapter 3, more specifically in Section 3.3, was successfully implemented by using the 26 cepstrum coefficient to shape information to be fed to the first layer of neurons. With the MFCC, the signal used as an input is converted to the frequency domain and distributed evenly on a scale mimicking the human ear. Without this step, the neural network wouldn't have been able to receive any input.

For the main part, which consisted of the design and development of the neural network model, a reference model was originally used for understanding and testing. The silicon-valley-data-science model with the simple LSTM neural network proved to be a strong starting point for our own neural networks. As it was used to get acquainted with all the different dependencies and intricacies that these systems possess. By reverse engineering the reference model, we were able to construct a series of more complex and better-performing networks.

By trial and error, the first fully functional model that outperformed the silicon-valley-data-science model was described in Section 6.1. The addition of fully connected layers both before and after the core LSTM layers proved to be an efficient way of reducing the word error rate across all the testing range, clearly seen in Figure 6.5. A further analysis of the state-of-the-art neural networks developed specifically for understanding speech revealed that a BiRNN could prove to be a better solution to the classical stacking of LSTM layers. From the idea that hidden vectors can compute previous and future states, a second model was pursued, having a BiRNN layer as its core. Seen in Figure 6.3, the best performing model is shown as a combination of FC and BiRNN.

Both models, LSTM and BiRNN were trained in the English language, by using the LibriSpeech set. To achieve this goal, a large number of parameters had to be tuned to ensure that the network didn't diverge during learning. As there is no predetermined way to set the parameters, training was done on an iterative process and the results were documented in Appendix C.

The next step was to distribute the speech recognition (SR) system. The server was established on the computer which had the video card and where the neural network was designed. This was done in order to assure that the server had enough processing power to complete the tasks sent by the users. Via the LAN network, any client that has access to the SR system can send a .wav file and receive a written text file containing the transcription of their voice.

9.1 Future Work

For future development, a series of topics can be pursued for the betterment of the DSR system. One of the key ideas pointed out in the report was that training data is crucial for the system, and as such, a first addition should constitute in the creation of even larger datasets. While a larger set of data will prove beneficial, sets that emulate real-life speech, such as speech in a noisy environment or multiple speakers at the same time will greatly benefit the learning curve of any neural network. The scope of this project was also limited by the hardware available at the time of this research paper, with such considerations, more powerful hardware will prove instrumental in reduc-

ing the processing time and increasing the efficiency of writing and developing better neural networks. Due to time constraints, a balance between the time that the network was training and time the models were developed had to be maintained. Moreover, data sets should be able to run for increased amounts of time to reduce the word error rate even further.

A major goal that can be achieved by building upon the results of this project is the creation of an information retrieval (IR) system. As data is returned in text format to the client, an IR system can search a predefined database, or use a web crawler to access specific web pages and return meaningful snippets of information related to the predetermined interests of the user.

Another improvement would target the distribution part of the system. TensorFlow Serving is a flexible, high-performance serving system for machine learning models. It is designed for production environments, and it provides an easy way to deploy new algorithms and experiments, without changing the server architecture, illustrated in 9.1. It would provide a great option for testing different models efficiently.

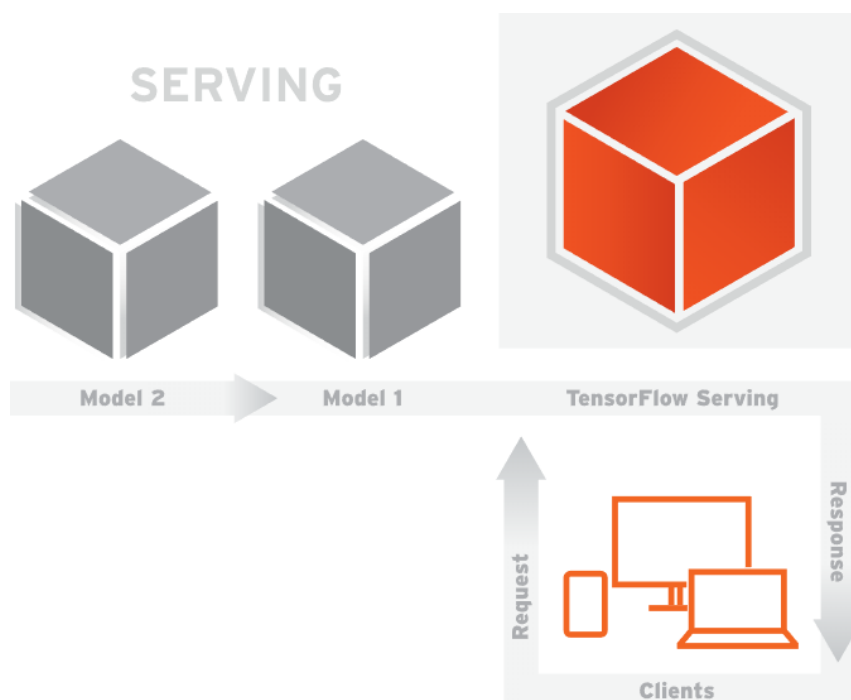


Figure 9.1: TensorFlow Serving [27].

Chapter 10

Conclusion

Based on the findings described in the previous chapter, the success criteria that we set were fully realized during the project period. In order to create the distributed speech recognition system, extensive research was done in the field of artificial neural networks, in order to create new neural networks that can provide adequate results. Based on the testing overtaken for all models detailed in this paper, the most efficient model is a fully functional network which acts as an improvement in all aspects compared to the starting point of the research done by us.

The findings show how a neural network approach to the problem of speech recognition returns concrete results and increased performances. This brings the idea of a building a non-intrusive way of interacting with computers one step closer to reality, and being considered a standard in everyday life.

Bibliography

- [1] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from [tensorflow.org](https://www.tensorflow.org/). 2015. URL: <https://www.tensorflow.org/>.
- [2] Bhushan C Kamble. "Speech Recognition Using Artificial Neural Network—A Review". In: ().
- [3] John S., Stefan B., and Kamran A. *Distributed Speech Recognition System*. <https://github.com/epiicme/Mavericks>. 2017.
- [4] "Specifications". In: (). URL: <https://www.geforce.co.uk/hardware/desktop-gpus/geforce-gtx-1070/specifications>.
- [5] R. Callan. *Artificial Intelligence*. Macmillan Education UK, 2003. ISBN: 9780333801369. URL: <https://books.google.dk/books?id=S1N0QgAACAAJ>.
- [6] Darrell Etherington. *Alexa*. <https://techcrunch.com/2014/11/06/amazon-echo/>. Accessed: 2017-10-30.
- [7] *Cortana*. <https://support.microsoft.com/en-us/help/17214>. Accessed: 2017-10-30.
- [8] Arie Ogranovich. *Speed*. <https://www.quora.com/Can-we-type-in-English-faster-than-we-speak>. Accessed: 2017-10-30.
- [9] Md Sahidullah and Goutam Saha. "Design, analysis and experimental evaluation of block based transformation in MFCC computation for speaker recognition". In: *Speech Communication* 54.4 (2012), pp. 543–565.
- [10] MATTHEW RUBASHKIN. *RNN-Tutorial*. <https://github.com/silicon-valley-data-science/RNN-Tutorial>. 2017.
- [11] Matt Mollison Matthew Rubashkin. "Building, training and improving on existing reccurent neural networks". In: (Mar. 2017).
- [12] Ruslan Leont'evich Stratonovich. "Conditional markov processes". In: *Theory of Probability & Its Applications* 5.2 (1960), pp. 156–178.
- [13] "Neural Networks CTC". In: (2017). URL: https://www.tensorflow.org/api_guides/python/nn#Connectionist_Temporal_Classification CTC_.

- [14] “Gram-CTC: Automatic Unit Selection and Target Decomposition for Sequence Labelling”. In: (2017). URL: <https://arxiv.org/abs/1703.00096>.
- [15] Santosh K Gaikwad, Bharti W Gawali, and Pravin Yannawar. “A review on speech recognition technique”. In: *International Journal of Computer Applications* 10.3 (2010), pp. 16–24.
- [16] Roberto Togneri, MD Alder, and Yianni Attikiouzel. “Speech processing using artificial neural networks”. In: (1990), pp. 304–309.
- [17] Ivan Nunes Da Silva et al. *Artificial Neural Networks: A Practical Course*. Springer, 2016.
- [18] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-term Memory”. In: 9 (Dec. 1997), pp. 1735–80.
- [19] Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. “Learning to Forget: Continual Prediction with LSTM”. In: *Neural Computation* 12 (1999), pp. 2451–2471.
- [20] “LSTM”. In: (2015). URL: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>.
- [21] Alex Graves, Santiago Fernández, and Jürgen Schmidhuber. “Bidirectional LSTM networks for improved phoneme classification and recognition”. In: *Artificial Neural Networks: Formal Models and Their Applications–ICANN 2005* (2005), pp. 753–753.
- [22] Alex Graves, Navdeep Jaitly, and Abdel-rahman Mohamed. “Hybrid speech recognition with deep bidirectional LSTM”. In: (2013), pp. 273–278.
- [23] Martin Görner. “Tensorflow and deep learning - without a PhD”. In: (2016). URL: <https://codelabs.developers.google.com/codelabs/cloud-tensorflow-mnist/#9>.
- [24] *Estimator*. https://www.tensorflow.org/get_started/get_started. Accessed: 2017-11-15.
- [25] *DeepSpeech*. <https://github.com/mozilla/DeepSpeech>. 2017.
- [26] “OpenVPN”. In: (2017). URL: <https://openvpn.net/>.
- [27] “TensorFlow Serving”. In: (2017). URL: <https://www.tensorflow.org/serving>.

Chapter 11

Appendix

Appendix A

MATLAB

A.1 Waveform, FFT and Spectrogram Script

```
1 %-----
2 %Loading the "Hello" sample
3 subplot(3,1,1);
4 [y,Fs] = audioread('microphone-results.wav');
5 sound(y,Fs);
6     dt = 1/Fs;
7     t = 0:dt:(length(y)*dt)-dt;
8     %figure();
9     plot(t,y); xlabel('Seconds'); ylabel('Amplitude');
10    title('Hello waveform');
11 %-----
12 %Creating the FFT of the signal
13 subplot(3,1,2);
14 N = length(y);
15 NEFT = 2^nextpow2(N)
16 Xk=fft(y, NEFT)/N; % perform N-point DFT of signal
17 fa = 8000/2*linspace(0,1,NEFT/2+1);
18 %figure();
19 plot(fa, 2*abs(Xk(1:NEFT/2+1)));
20 title('FFT Spectrum Estimates / Power Spectral Density');
21 xlabel('Frequency Index'),
22 ylabel('Amplitude');
23 %-----
24 % Creating the Spectrogram
25 %figure();
26 subplot(3,1,3);
27 s = spectrogram(y);
28 spectrogram(y,'yaxis')
29 %-----
```

Listing A.1: MATLAB script.

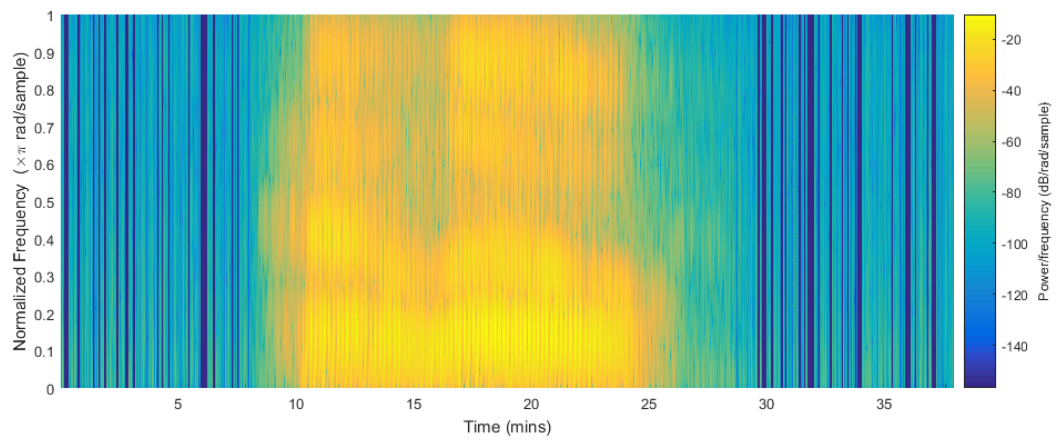


Figure A.1: Spectrogram generated from FFT.

Appendix B

Batch Size, Dropout and Learning Rate Comparison Data

B.1 Batch Size Test

Final table results

Table B.1: Test error rate results.

Tests	Value	Epoch	Duration
Test1 -●-	0.1308	25.00	0s
Test2 -●-	0.000	25.00	0s
Test5 -●-	0.000	25.00	0s

Table B.2: Training error rate results.

Tests	Value	Epoch	Duration
Test1 -●-	0.1148	24.00	1h 26m 13s
Test2 -●-	9.5073e-4	24.00	18m 50s
Test5 -●-	5.1858e-4	24.00	14m 50s

Table B.3: Validation error rate results.

Tests	Value	Epoch	Duration
Test1 -●-	0.1608	23.00	1h 15m 58s
Test2 -●-	0.05887	23.00	17m 21s
Test5 -●-	0.05418	23.00	13m 50s

Table B.4: Training average loss results.

Tests	Value	Epoch	Duration
Test1 -●-	1.639	24.00	1h 26m 13s
Test2 -●-	0.06023	24.00	18m 50s
Test5 -●-	0.01988	24.00	14m 50s

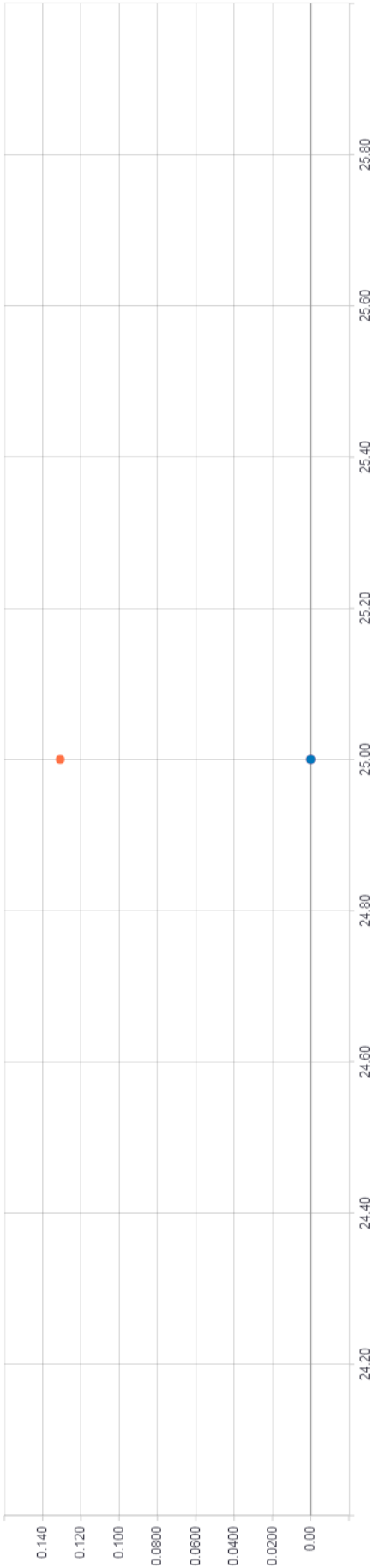


Figure B.1: Test error rate.

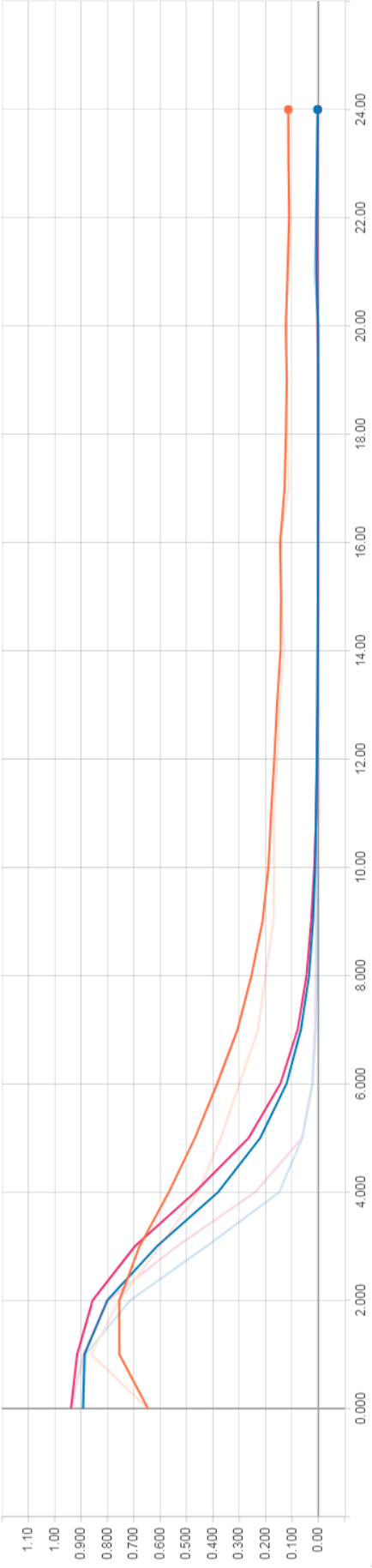


Figure B.2: Training error rate.

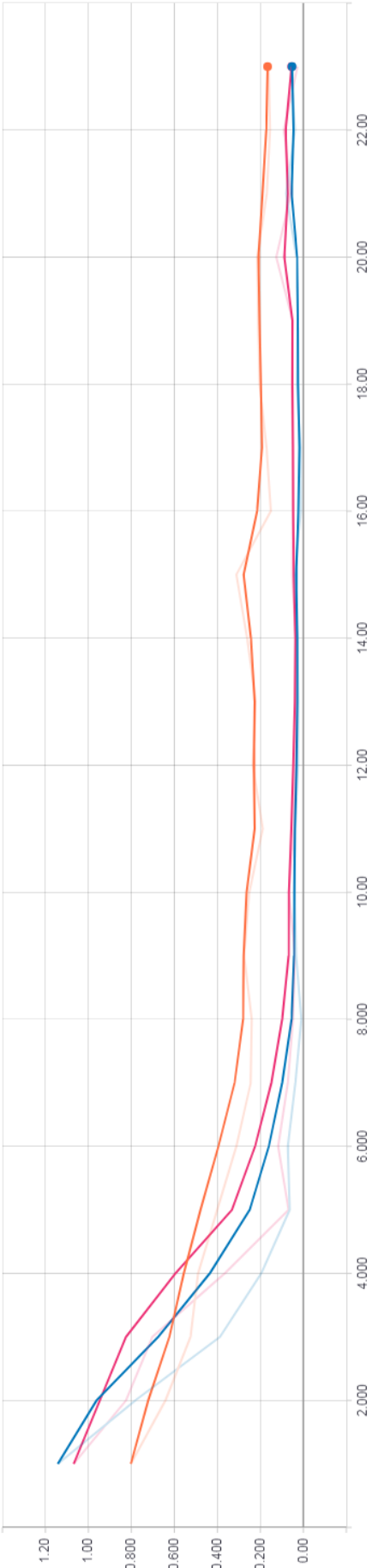


Figure B.3: Validation error rate.

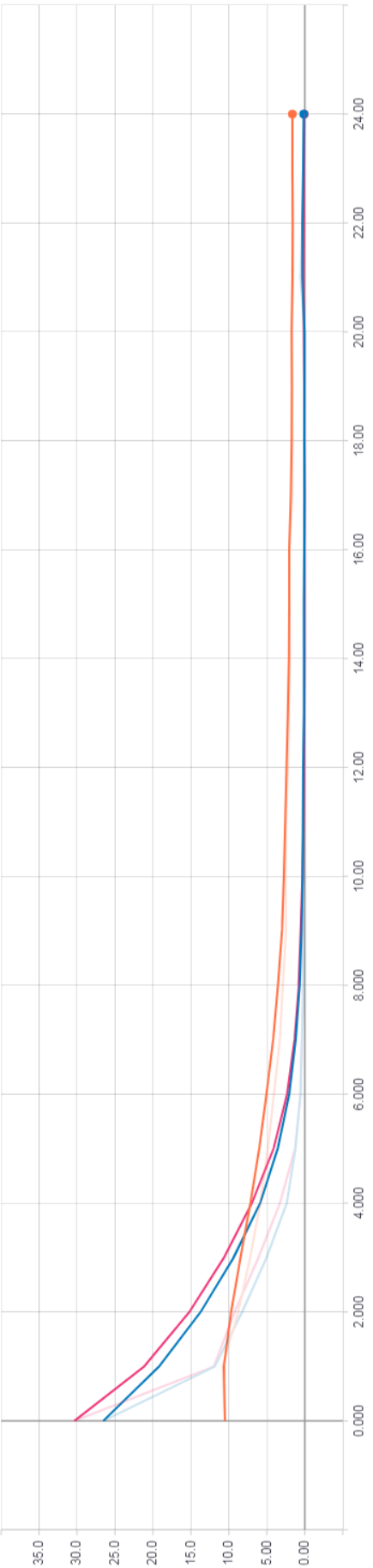


Figure B.4: Training average loss.

B.2 Dropout Test

Final table results

Table B.5: Test error rate results.

Tests	Value	Epoch	Duration
Test2 -●-	0.000	25.00	0s
Test3 -●-	0.030	25.00	0s
Test4 -●-	0.020	25.00	0s

Table B.6: Training error rate results.

Tests	Value	Epoch	Duration
Test2 -●-	9.5973e-4	24.00	18m 50s
Test3 -●-	0.000	24.00	18m 5s
Test4 -●-	6.3331e-3	24.00	18m 10s

Table B.7: Validation error rate results.

Tests	Value	Epoch	Duration
Test2 -●-	0.05887	23.00	17m 21s
Test3 -●-	0.05161	23.00	16m 36s
Test4 -●-	0.02366	23.00	16m 35s

Table B.8: Training average loss results.

Tests	Value	Epoch	Duration
Test2 -●-	0.06023	24.00	18m 50s
Test3 -●-	3.5483e-4	24.00	18m 5s
Test4 -●-	0.09217	24.00	18m 10s

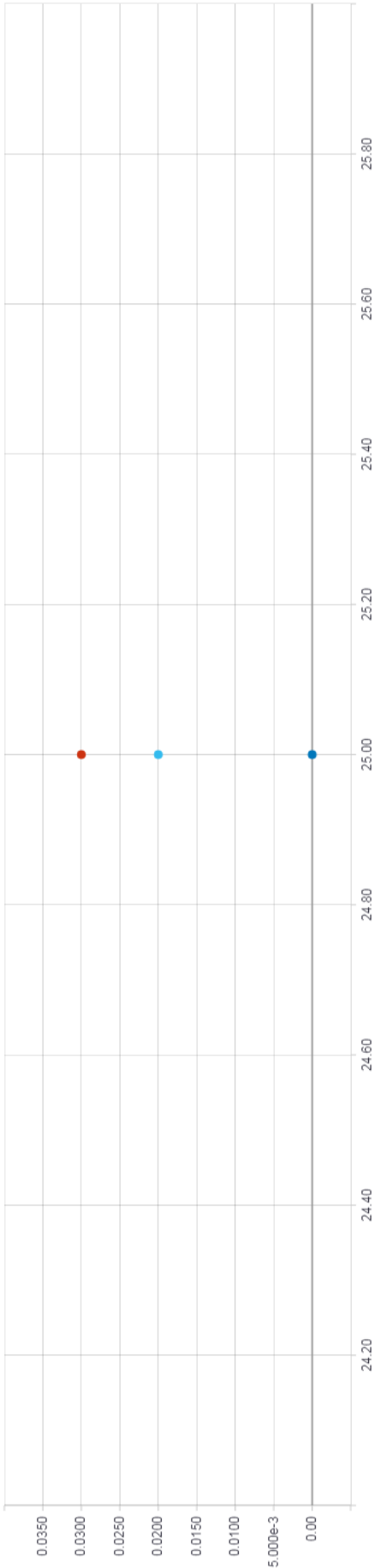


Figure B.5: Test error rate.

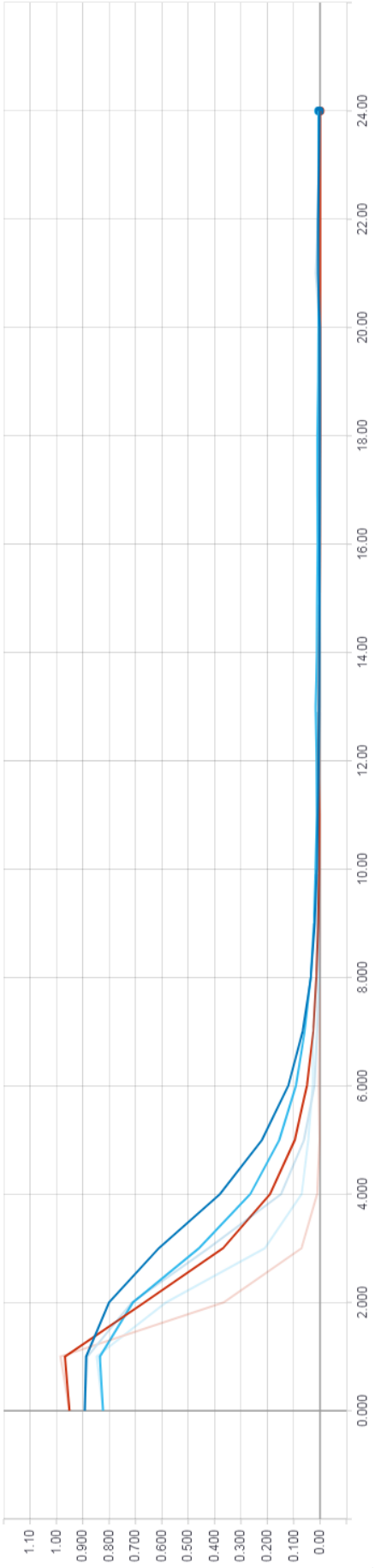


Figure B.6: Training error rate.

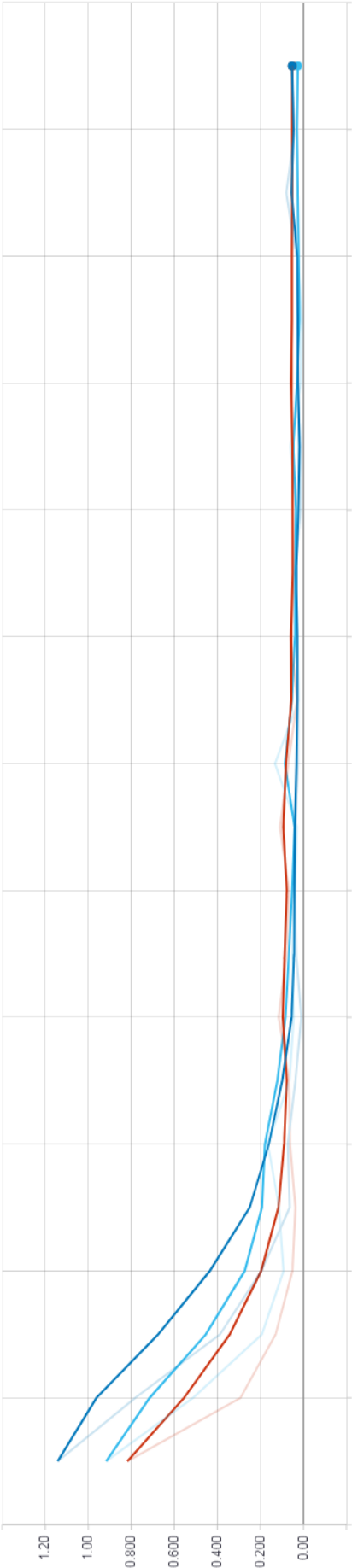


Figure B.7: Validation error rate.

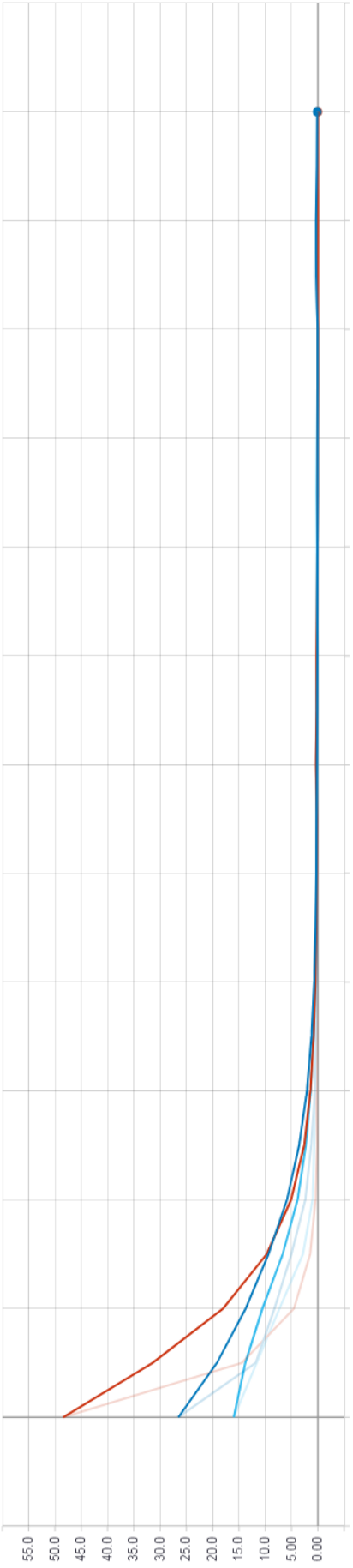


Figure B.8: Training average loss.

B.3 Learning Rate Test

Final table results

Table B.9: Test error rate results.

Tests	Value	Epoch	Duration
Test5 -●-	0.000	25.00	0s
Test6 -●-	0.8992	25.00	0s

Table B.10: Training error rate results.

Tests	Value	Epoch	Duration
Test5 -●-	5.1858e-4	24.00	14m 50s
Test6 -●-	0.9188	24.00	12m 29s

Table B.11: Validation error rate results.

Tests	Value	Epoch	Duration
Test5 -●-	0.02661	23.00	13m 50s
Test6 -●-	1.152	23.00	11m 29s

Table B.12: Training average loss results.

Tests	Value	Epoch	Duration
Test5 -●-	0.01988	24.00	14m 50s
Test6 -●-	12.56	24.00	12m 29s

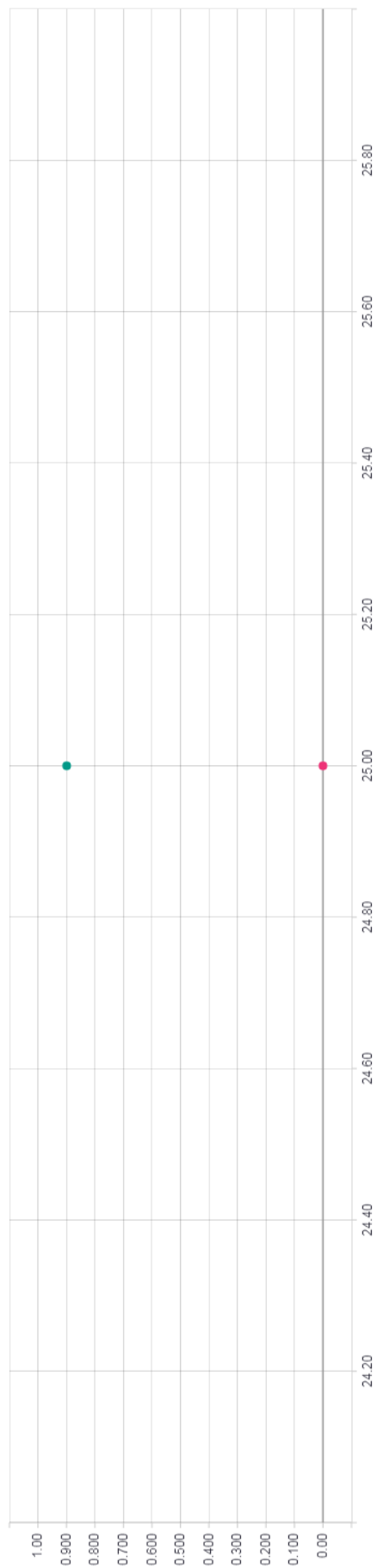


Figure B.9: Test error rate.

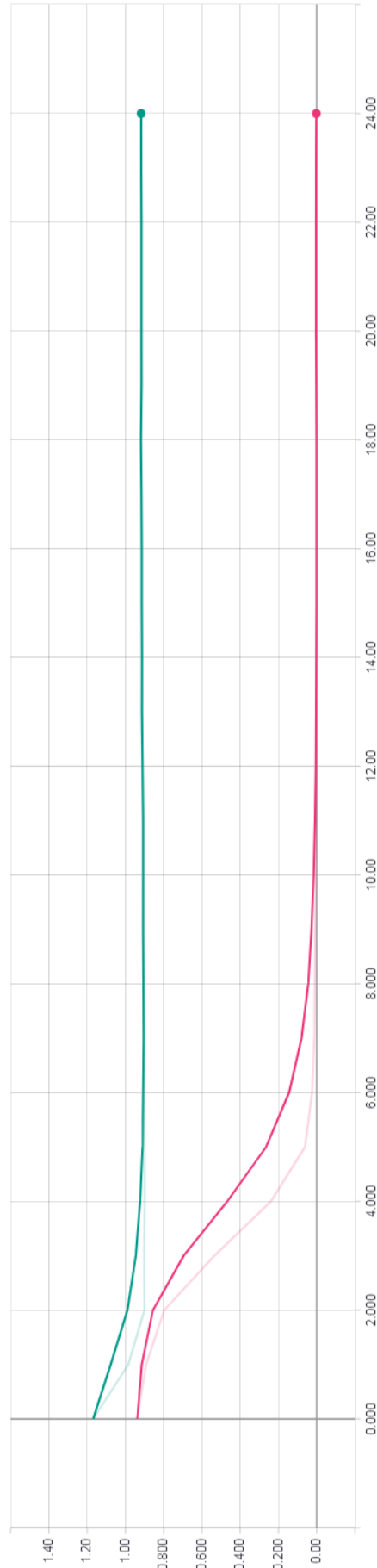


Figure B.10: Training error rate.

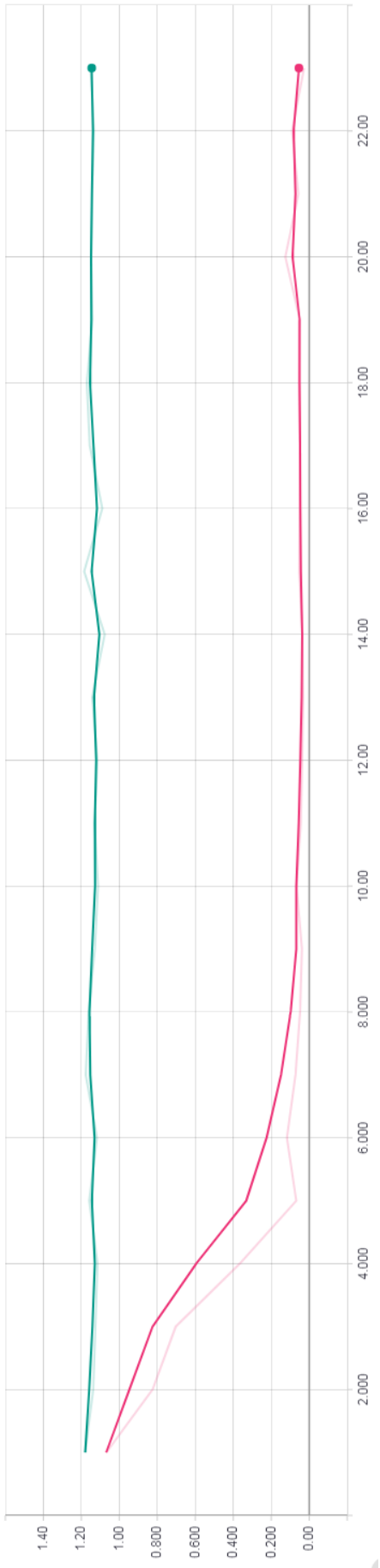


Figure B.11: Validation error rate.

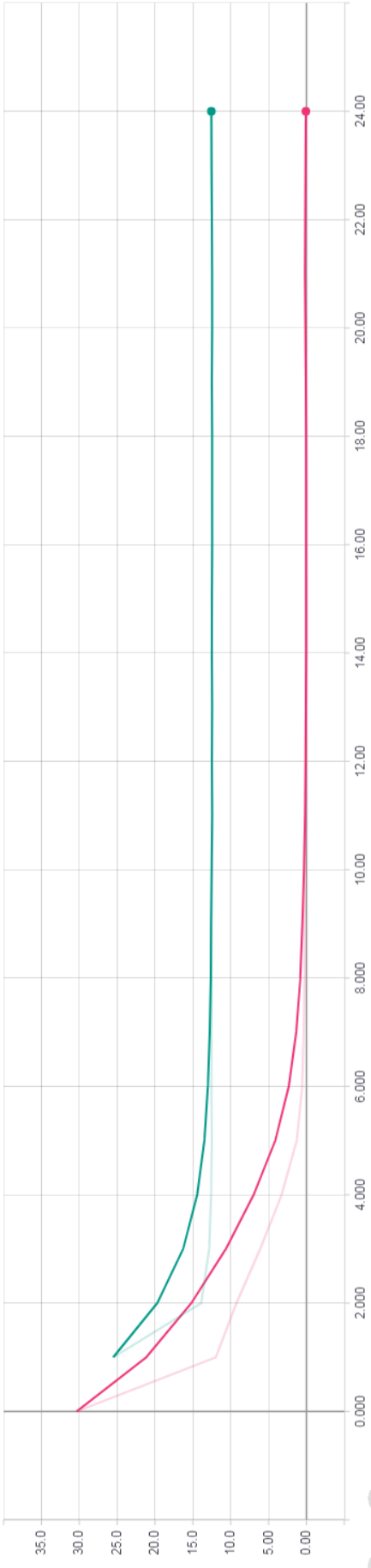


Figure B.12: Training average loss.

Appendix C

Model Development Comparison Data

Table C.1: Test error rate results.

Models	Value	Epoch	Duration
Model1 -●-	0.200	25.00	0s
Model2 -●-	0.080	25.00	0s
Model3 -●-	0.000	25.00	0s

Table C.2: Training error rate results.

Models	Value	Epoch	Duration
Model1 -●-	2.0887e-3	24.00	7m 46s
Model2 -●-	5.1642e-3	24.00	9m 47s
Model3 -●-	0.000	24.00	12m 50s

Table C.3: Validation error rate results.

Models	Value	Epoch	Duration
Model1 -●-	0.1669	23.00	7m 9s
Model2 -●-	0.2030	23.00	9m 8s
Model3 -●-	0.01398	23.00	11m 49s

Table C.4: Training average loss results.

Models	Value	Epoch	Duration
Model1 -●-	0.04573	24.00	7m 46s
Model2 -●-	0.1608	24.00	9m 47s
Model3 -●-	1.2939e-3	24.00	12m 50s

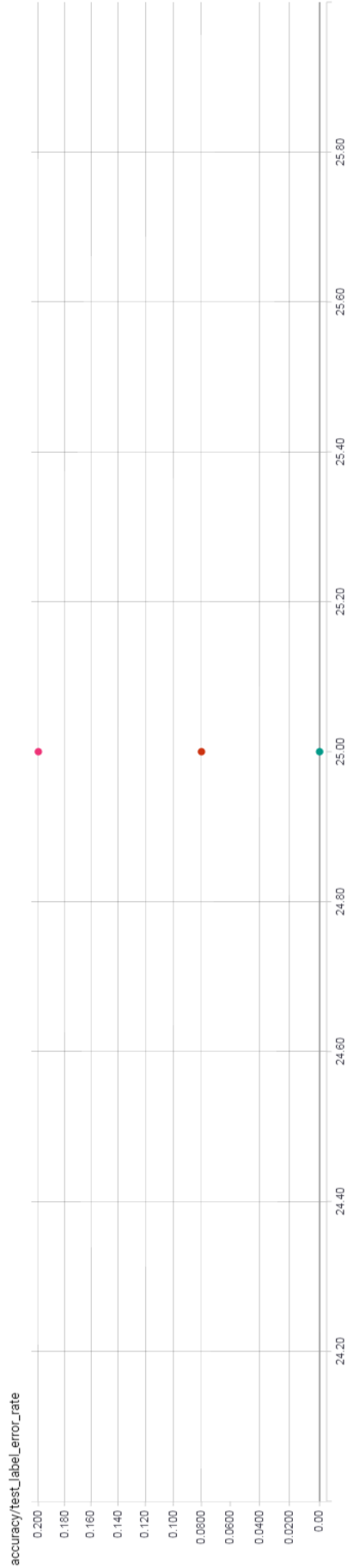


Figure C.1: Test error rate.

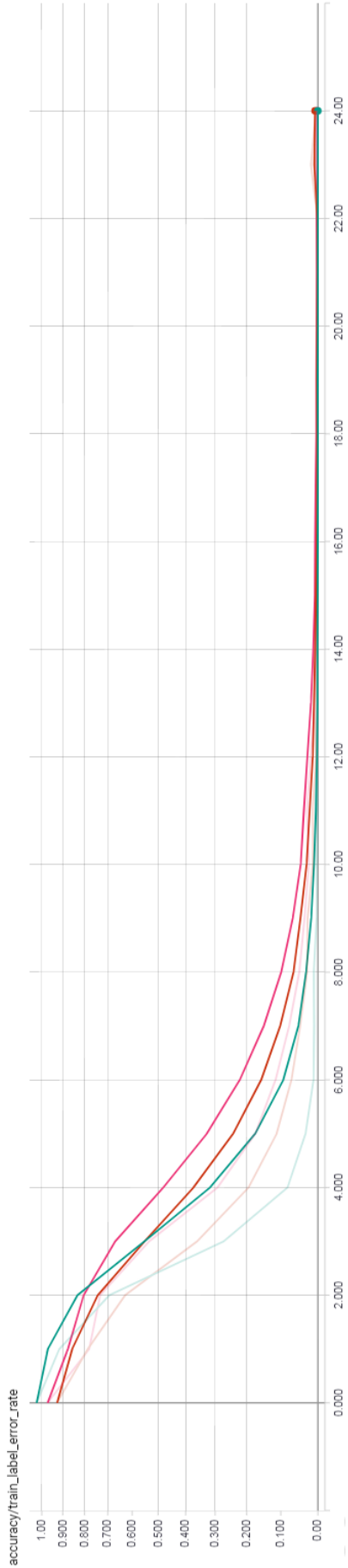


Figure C.2: Training error rate.

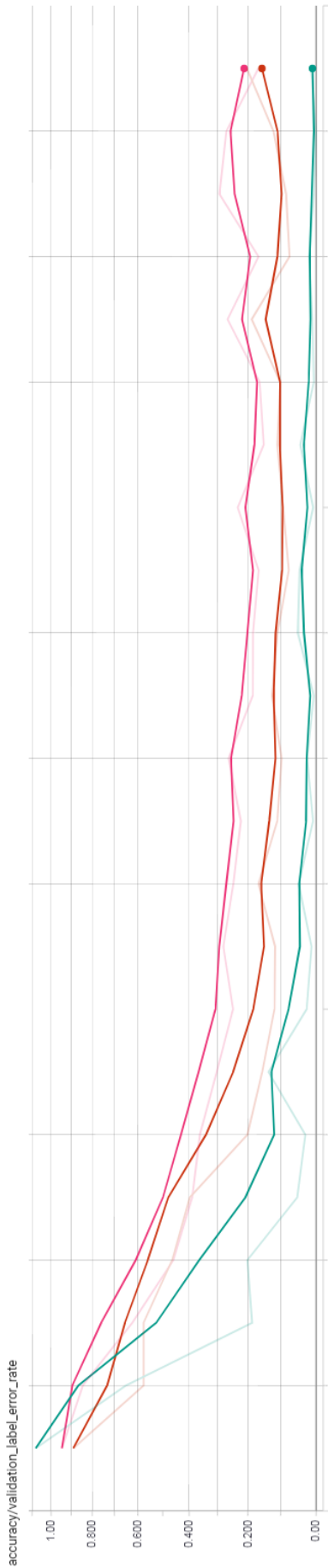


Figure C.3: Validation error rate.

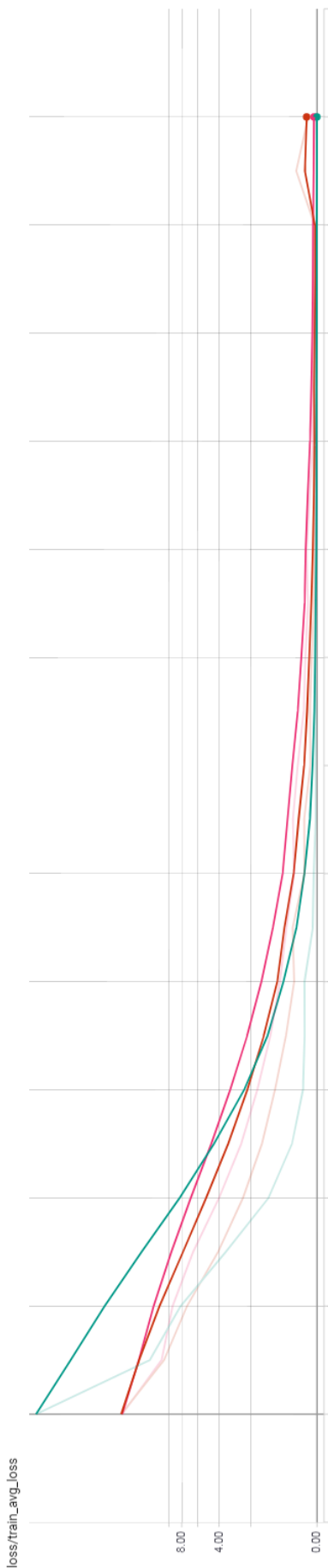
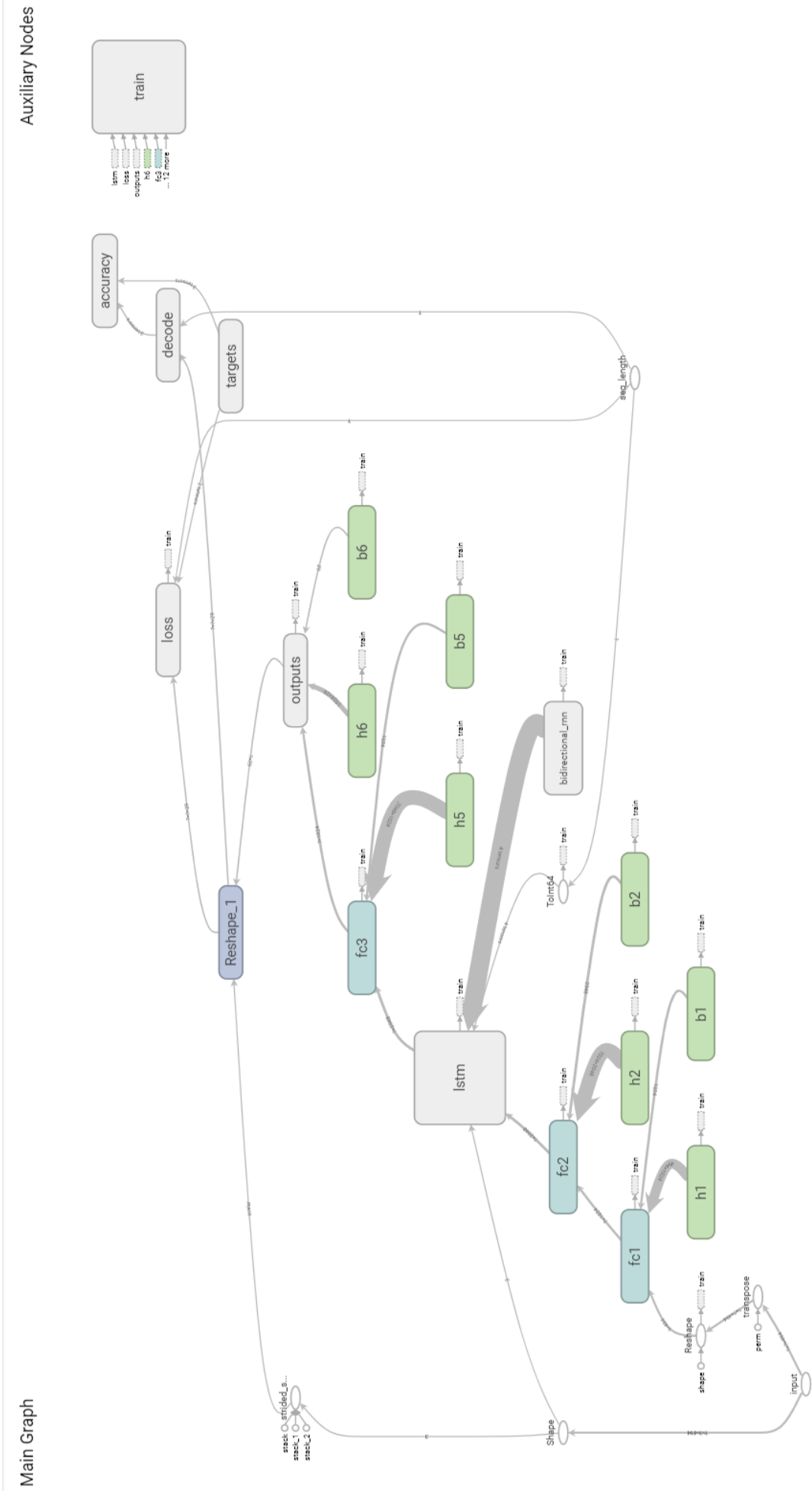


Figure C.4: Training average loss.



Appendix D

Data Mining Code

D.1 FlacToWavPy

```
1 # Designed to work with the OpenSLR LibriSpeech ASR corpus (which is in .flac)
2 # Requires pydub
3 # Script that goes in a folder and all subfolders and converts .flac files into .wav files. Can be
  imported as module or run directly from command line.
4 # Input format on cmd line: "python FlacToWavPy.py 'source' 'dest'"
5 # Source and dest(without apostrophes) are absolute paths to the folder containing the .flac files and
  the folder to put the converted .wav files respectively
6
7 import os
8 import sys
9 from pydub import AudioSegment
10
11 class raiseExcept(Exception):
12     pass
13
14 def flac_check(input_name):
15     """ Function that checks if a file is .flac or not. Returns True or False (bool)
16         Parameter: 'input_name' string that has the name of the file (with format). It may work with
17         absolute path and name (maybe) """
18
19     _, file_extension = os.path.splitext(input_name)
20
21     if file_extension == '.flac':
22         return True
23     else:
24         return False
25
26 def flac_to_wav(input_flac, input_name, dest):
27     """ Function that converts a .flac to a .wav file. Parameter: 'input_flac' – string abs path to a
28     .flac,
29     'input_name' – string of only the name of the .flac (with format), 'dest' string abs path to
30     destination folder (with a '/' in the end!) """
```

```

28
29 audio_file = AudioSegment.from_file(input_flac)
30 temp_name = dest
31 temp_name += input_name
32 temp_name = temp_name[:len(temp_name)-4]
33 temp_name += ".wav"
34 audio_file.export(temp_name)
35 print(("File %s conv success!")%(input_name))
36
37 def folder_navigator_convert(source, dest):
38     """ Function that goes in a folder and all subfolders and converts .flac files into .wav files.
39         Parameters: 'source' – string, abs path to the root folder, 'dest' – string, abs path to dest
40         folder (both must be with a '/' at the end!) """
41
42     for root, dirs, files in os.walk(source):
43         for name in files:
44             abs_path = os.path.join(root,name)
45
46             if '\\\\' in abs_path:
47                 abs_path = abs_path.replace("\\\\",'/')
48
49             if flac_check(name):
50                 flac_to_wav(abs_path,name,dest)
51             else:
52                 print(("File %s ignored!")%(name))
53
54 if __name__ == "__main__":
55
56     try:
57         numb_of_args=len(sys.argv)-1
58
59         if not numb_of_args == 2:
60             raise raiseExcept()
61
62     except(raiseExcept):
63         print("Number of input parameters wrong! Correct format: 'source' 'dest'")
64         sys.exit()
65
66     try:
67         #source_path = 'C:\\\\Users\\Mavericks\\Documents\\GitHub\\Mavericks\\coding\\PyTests\\
68         FolderNavTest\\Testing_folder' //these two could be used instead of sys.argv[1] and sys.
69         argv[2]
70         #dest_path = 'C:\\\\Users\\Mavericks\\Documents\\GitHub\\Mavericks\\coding\\PyTests\\
71         FolderNavTest\\Results_folder' // in order to run the code in an IDE or similar
72         source_path = sys.argv[1]
73         dest_path = sys.argv[2]
74
75         if '\\\\' in source_path:
76             source_path = source_path.replace("\\\\",'/')

```

```

74     if '\\\\' in dest_path:
75         dest_path = dest_path.replace('\\\\', '/')
76
77     if not os.path.isdir(source_path):
78         raise raiseExcept()
79
80     if not os.path.isdir(dest_path):
81         raise raiseExcept()
82
83     if source_path[len(source_path)-1:] is not '/':
84         source_path+='/'
85
86     if dest_path[len(dest_path)-1:] is not '/':
87         dest_path+='/'
88
89     folder_navigator_convert(source_path,dest_path)
90
91 except(raiseExcept):
92     print("Wrong parameter format! Correct format: 'source' 'dest'")

```

Listing D.1: FlacToWavPy code.

D.2 txtFileMaker

```

1  # Designed to work with the OpenSLR LibriSpeech ASR corpus (which is in .flac)
2  # Script that goes in a folder and all subfolders and parses one .txt file containg paragraphs to
3  # multiple .txt files corresponding to their .wav files
4  # The result would be: for every .wav with the standard OpenSLR name, one .txt file with the same
5  # name as the .wav (audio corresponds to the paragraph)
6  # Input format on cmd line: "python txtFileMaker.py 'source' 'dest'"
7  # Source and dest(without apostrophes) are absolute paths to the folder containing the unsplit .txt
8  # files and the folder where the converted .flac to .wav files are respectively
9
10 import os
11 import sys
12
13 class raiseExcept(Exception):
14     pass
15
16 def txt_check(input_name):
17     """ Function that checks if a file is .txt or not. Returns True or False (bool)
18     Parameter: 'input_name' string, that has the name of the file (with format). It may work with
19     absolute path and name (maybe) """
20
21     _, file_extension = os.path.splitext(input_name)
22
23     if file_extension == '.txt':
24         return True
25     else:
26         return False

```

```

23
24 def txt_maker(input_file,input_name,dest):
25     """ Function that splits a .txt file into multiple .txt files so that the .wav files and .txt
26         files are 1 to 1 pairs.
27         Parameter: 'input_file' – string abs path to a .txt ,
28         'input_name' – string of only the name of the .txt (with format),
29         'dest' string abs path to destination folder (with a '/' in the end!) """
30
31     new_file_counter = 0
32     temp_len = len(input_name)
33     temp_name = input_name[:temp_len-10]
34
35     fo = open (input_file,"r")
36
37     str_temp = fo.read()
38
39     char_to_look_for = " "
40     ctrl_seq_pos = str_temp.find(temp_name)
41     first_space = str_temp.find(char_to_look_for)
42
43     while ctrl_seq_pos != -1:
44
45         temp_start_content = first_space+1
46         new_file_name = dest
47         new_file_name += str_temp[ctrl_seq_pos:first_space] + ".txt"
48
49         ctrl_seq_pos = str_temp.find(temp_name,ctrl_seq_pos+20)
50
51         new_fo = open (new_file_name,"w")
52         new_file_counter+=1
53
54         if ctrl_seq_pos != -1:
55             new_content = str_temp[temp_start_content:ctrl_seq_pos-1]
56             first_space = str_temp.find(char_to_look_for,ctrl_seq_pos)
57         else:
58             new_content = str_temp[temp_start_content:len(str_temp)] # could be
59             problematic, maybe
60
61         new_fo.write(new_content)
62
63         new_fo.close()
64
65     print(("File %s parsed successfully into %i .txt files !") % (input_name, new_file_counter))
66
67     fo.close()
68
69 def add_numb_of_files(input):
70     return input+1
71
72 # 0 – wav, 1 – txt

```

```

71 def numb_of_files(input,wav_or_txt,count):
72     ''' Function that counts number of txt files '''
73     if wav_or_txt == 0:
74         if txt_check(input):
75             count = add_numb_of_files(count)
76
77     elif wav_or_txt == 1:
78         if not txt_check(input):
79             count = add_numb_of_files(count)
80
81     return count
82
83 def folder_navigator_txt(source, dest):
84     ''' Function that goes in a folder and all subfolders and splits .txt files into multiple .txt
85         files so that the .wav files and .txt files are 1 to 1 pairs.
86         Parameters: 'source' – string, abs path to the root folder, 'dest' – string, abs path to dest
87         folder (both must be with a '/' at the end!) '''
88
89     for root, dirs, files in os.walk(source):
90         for name in files:
91             abs_path = os.path.join(root,name)
92
93             if '\\\\' in abs_path:
94                 abs_path = abs_path.replace("\\\\", '/')
95
96             if txt_check(name):
97                 txt_maker(abs_path,name,dest)
98
99 if __name__ == "__main__":
100     try:
101         numb_of_args=len(sys.argv)-1
102
103         if not numb_of_args == 2:
104             raise raiseExcept()
105
106     except(raiseExcept):
107         print("Number of input parameters wrong! Correct format: 'source' 'dest'")
108         sys.exit()
109
110     try:
111         #source_path = 'C:\\\\Users\\Mavericks\\Documents\\GitHub\\Mavericks\\coding\\PyTests\\
112         FolderNavTest\\Testing_folder' //these two could be used instead of sys.argv[1] and sys.
113         argv[2]
114         #dest_path = 'C:\\\\Users\\Mavericks\\Documents\\GitHub\\Mavericks\\coding\\PyTests\\
115         FolderNavTest\\Results_folder' // in order to run the code in an IDE or similar
116
117         source_path = sys.argv[1]
118         dest_path = sys.argv[2]

```

```
116     if '\\\\' in source_path:
117         source_path = source_path.replace('\\\\','/')
118
119     if '\\\\' in dest_path:
120         dest_path = dest_path.replace('\\\\','/')
121
122     if not os.path.isdir(source_path):
123         raise raiseExcept()
124
125     if not os.path.isdir(dest_path):
126         raise raiseExcept()
127
128     if source_path[len(source_path)-1:] is not '/':
129         source_path+='/'
130
131     if dest_path[len(dest_path)-1:] is not '/':
132         dest_path+='/'
133
134     folder_navigator_txt(source_path,dest_path)
135
136 except(raiseExcept):
137     print("Wrong parameter format! Correct format: 'source' 'dest'")
```

Listing D.2: txtFileMaker code.